

Stack — Fundamentals, Monotonic Stack and the Stack Patterns

What this is. A complete study guide for the **Stack** data structure, built from a guided reasoning-first session on stacks (fundamentals, Next/Previous Greater/Smaller, Car Fleet, Largest Rectangle in Histogram) and then **extended to cover everything the session did not**: implementations, parentheses and cancelling problems, expression evaluation, design problems (Min Stack, queue $\leftrightarrow$ stack), the full monotonic-stack family, area/contribution problems, and stacks as the call stack (iterative DFS and tree traversals).

The one idea to keep. A stack is useful when *the most recently encountered unresolved thing must be handled before older unresolved things*. Don't memorise "*Daily Temperatures = stack*". Ask: **What is waiting? What resolves it? Does the newest waiting element get resolved first?**

What the original session covered — and what this guide adds

Topic	Original session	This guide
What a stack is, LIFO, push/pop/peek, stack vs array / queue / deque, Python list as a stack, complexity	✓	✓ (+ diagrams, implementations from scratch, deque, LifoQueue, amortized cost, underflow)
The "waiting elements" mental model and the recognition checklist	✓	✓
Next Greater / Next Smaller / Previous Smaller / Previous Greater, invariant, index-vs-value, Form A vs Form B	✓ (concepts + templates)	✓ (+ traces, duplicates and strictness, amortized proof)
Car Fleet	✓ (idea + simulation, no code)	✓ full solution
Largest Rectangle in Histogram	✓ (idea, stopped before the code)	✓ full solution + traces
Parentheses & cancelling problems (Valid Parentheses, Longest Valid Parentheses, ...)	✗	✓ added
Expression evaluation (RPN, Basic Calculator I/II, Decode String, Simplify Path)	✗	✓ added
Design problems (Min Stack, Queue $\leftrightarrow$ Stack, Browser History, ...)	✗	✓ added
Daily Temperatures, Stock Span, Remove K Digits, circular arrays, Sum of Subarray Minimums, 132 Pattern ...	mentioned only	✓ solved
Maximal Rectangle, Trapping Rain Water	✗	✓ added
Asteroid Collision (stack simulation)	✗	✓ added
Stack as the call stack: iterative DFS, in/pre/post-order traversals	✗	✓ added
Monotonic deque (Sliding Window Maximum)	✗	✓ added

Topic	Original session	This guide
Templates, decision guide, pitfalls, complexity table, practice list, interview Q&A	partly	✓

More than 50 problems and implementations are solved in Python. Every program was run against brute-force solutions on thousands of random inputs, and the step tables are produced by running the algorithms.

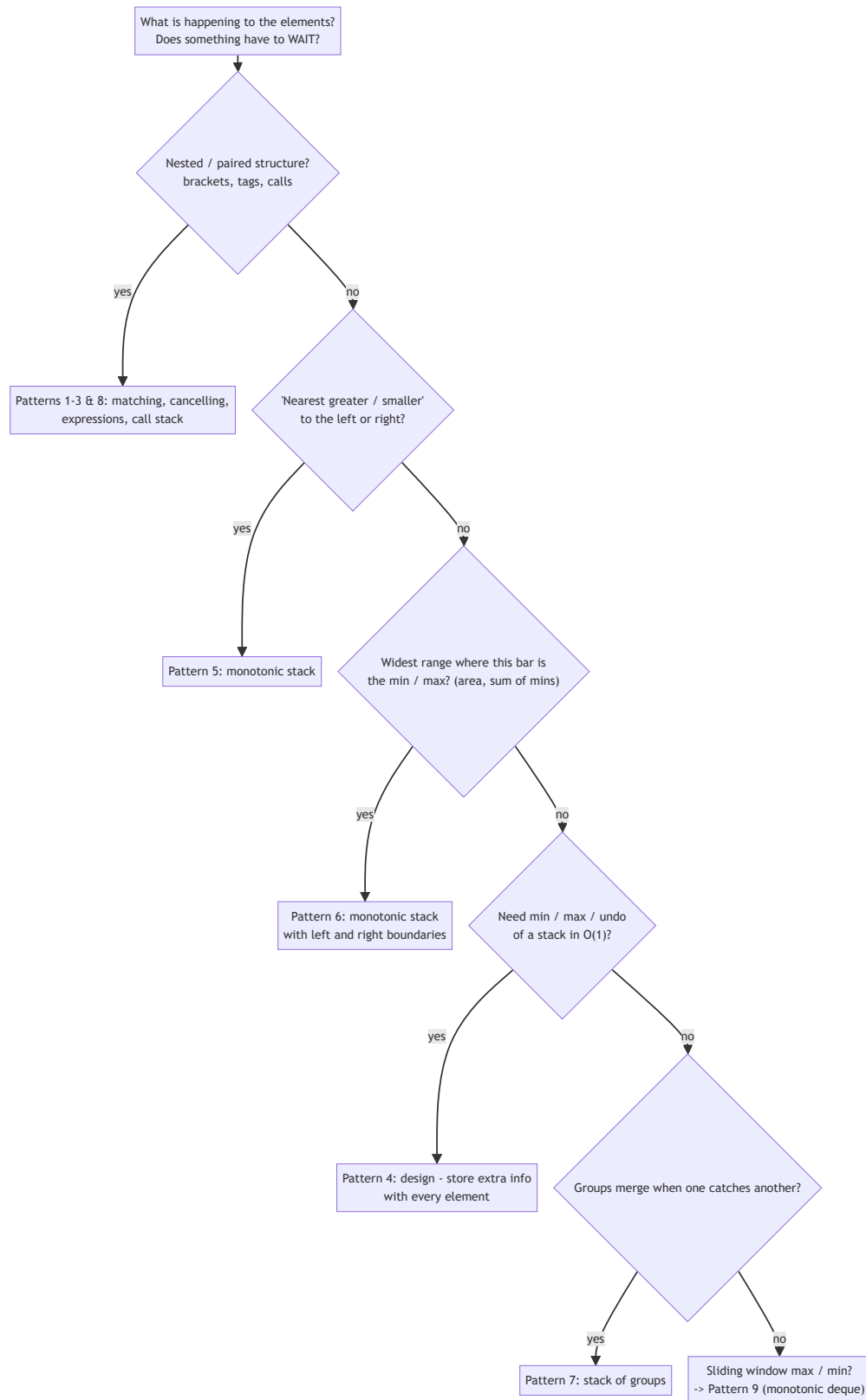
Corrections to the original session. Three points in the source needed fixing, each marked **Correction** below: (1) in the Car Fleet drill the car at position 8 was given time `0.66`, but $(10 - 8) / 2 = 1$ and the notes marked it "correct"; (2) the notes state the Next-Greater stack is *strictly* decreasing, but with the `>` pop rule equal values stay, so it is *non-increasing*; (3) the histogram walkthrough says the 2 "violates $5 < 6$ ", but the real reason a bar is popped is $2 < 6$ and $2 < 5$ (the arriving bar is *shorter than* the top).

Part 0 · Roadmap

0.1 The stack patterns at a glance

#	Pattern	What is "waiting"?	Typical problems
1	Matching / nesting	an opener waiting for its closer	Valid Parentheses, Longest Valid Parentheses, Min Remove to Make Valid, Score of Parentheses, Reverse Substrings
2	Cancelling	an item waiting to be cancelled by its neighbour	Remove Adjacent Duplicates, Backspace String Compare, Asteroid Collision
3	Expression evaluation	an operand / operator waiting for its partner	RPN, Basic Calculator I & II, Decode String, Simplify Path
4	Design with stacks	(auxiliary information stored with each element)	Min Stack, Max Stack, Queue from Stacks, Browser History
5	Monotonic stack	an element waiting for a greater/smaller one	Next Greater, Daily Temperatures, Stock Span, Remove K Digits, 132 Pattern
6	Monotonic stack — areas & contributions	a bar waiting for a shorter bar	Largest Rectangle, Maximal Rectangle, Trapping Rain Water, Sum of Subarray Minimums
7	Stack of groups	a fleet waiting to be caught / merged	Car Fleet
8	Call stack / DFS	a node waiting for its children to finish	iterative DFS, tree traversals, flood fill
9	Monotonic deque <i>(related)</i>	window elements that can still become the maximum	Sliding Window Maximum

0.2 First-glance decision tree

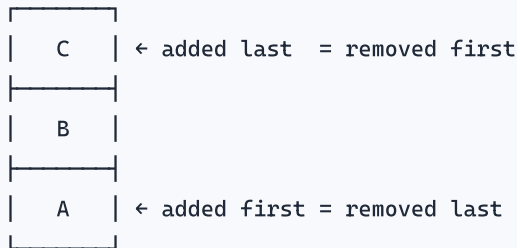


Part 1 · Stack Fundamentals

The important thing is not learning `push()` and `pop()`. The important question is: why would an algorithm want the most recently encountered thing to be handled first?*

1.1 What is a stack?

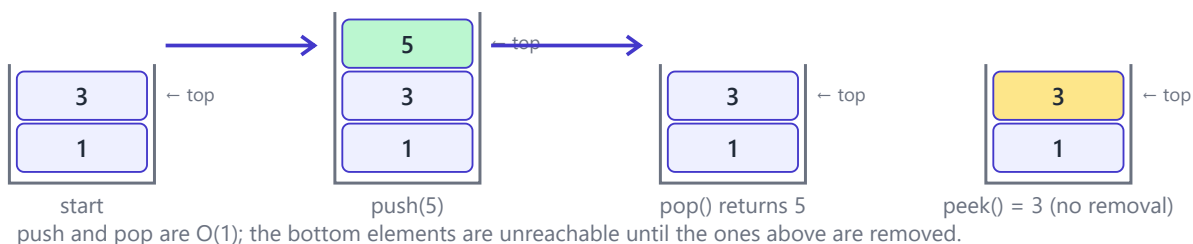
A stack is a data structure where **insertion and removal happen at the same end**. Imagine a pile of plates: you add a plate on top and you remove the top plate.



This is **LIFO — Last In, First Out**. Push A, push B, push C; the removal order is C → B → A.

1.2 The three fundamental operations

The three operations: push, pop, peek (everything happens at the TOP)



Operation	Meaning	Python list	Cost
push	put an item on top	<code>stack.append(x)</code>	$O(1)$ amortized
pop	remove and return the top	<code>stack.pop()</code>	$O(1)$
peek / top	look at the top without removing	<code>stack[-1]</code>	$O(1)$
is_empty	any items?	<code>not stack</code>	$O(1)$
size	how many items	<code>len(stack)</code>	$O(1)$

- **Underflow** — popping/peeking an empty stack. In Python this raises `IndexError`; always guard with `if stack` when the emptiness is possible.
- **Overflow** — pushing onto a full *fixed-capacity* stack (Python lists grow dynamically, so this only matters for fixed-size stacks or the *call stack*, where too-deep recursion gives `RecursionError` / stack overflow).

1.3 Why is LIFO useful for algorithms?

This is the first genuinely important concept. Consider $A \rightarrow B \rightarrow C$ where **C opens something that must be completely handled before we return to B**:

```
A
└─ B
   └─ C
```

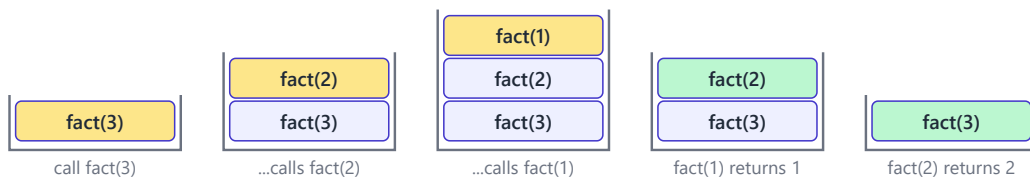
Enter A → Enter B → Enter C → Finish C → Return to B → Finish B → Return to A

The most recently entered thing is the first one we return to. That is why stacks appear in:

- nested brackets · nested function calls · recursion
- undo operations · expression evaluation · parsing
- DFS
- "nearest previous / next" problems
- elements waiting for a future element → **monotonic stack**

The common underlying idea: **something newer temporarily takes priority over something older**.

Recursion uses a stack: the call stack ($\text{fact}(3) = 3 * \text{fact}(2) = 3 * 2 * \text{fact}(1)$)



Newest call = top of the stack = the one that must finish first. An explicit stack in your own code plays the same role (DFS, undo, parsing).

1.4 Stack vs Array

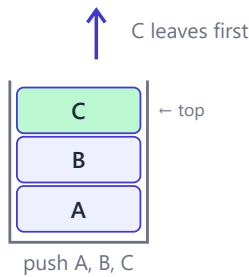
An **array** is a way of *storing* elements by position — you can read `array[0]`, `array[2]`, `array[3]` freely. A **stack** *restricts how you interact* with the collection: you can only reach the top; to reach `20` you must first remove `40` and `30`.

Array = storage structure with indexed access. Stack = an access discipline based on LIFO. Python commonly implements a stack *using* a list.

1.5 Stack vs Queue vs Deque

Stack (LIFO) vs Queue (FIFO)

Stack: last in, first out



Queue: first in, first out



Ask: do I need the **NEWEST** pending item (stack) or the **OLDEST** (queue)?

	Stack	Queue
Rule	LIFO	FIFO
First removed	most recent	oldest
Intuition	"go back to what I just did"	"process whoever has waited longest"
Examples	undo, nested structures, DFS	BFS, scheduling, buffering

A useful mental question: **"Do I care about the newest pending item or the oldest pending item?"** Newest → stack. Oldest → queue.

A **deque** (double-ended queue) allows adds/removes at *both* ends, so it can be used to implement a stack (use one end only), a queue, or both — it is more general. A stack conceptually restricts you to one end.

1.6 Representing a stack in Python

Choice	Push / pop	Notes
<code>list</code>	<code>append()</code> / <code>pop()</code>	the default; for DSA a list is enough
<code>collections.deque</code>	<code>append()</code> / <code>pop()</code>	$O(1)$ worst case (no amortised resizing); use when you also need the other end
<code>queue.LifoQueue</code>	<code>put()</code> / <code>get()</code>	thread-safe, but slow due to locking — not for algorithms
your own linked list	head insert / head remove	$O(1)$ worst case; no resizing; see below

```
stack = []
stack.append(2); stack.append(5); stack.append(7)    # top is now 7
x = stack.pop()                                     # x = 7, stack = [2, 5]
top = stack[-1]                                     # peek = 5
```

Complexity of a Python list used as a stack

Operation	Complexity
<code>append()</code>	$O(1)$ amortized
<code>pop()</code> from the end	$O(1)$

Operation	Complexity
<code>stack[-1]</code>	$O(1)$
<code>pop(0)</code>	$O(n)$ — every element shifts left
<code>insert(0, x)</code>	$O(n)$

Never use the front of a list as the top of a stack. `insert(0, x)` / `pop(0)` shift the whole array. Always use the **end**. (*Amortized* $O(1)$ for `append` : the list occasionally reallocates and copies, but spread over many appends the average cost per append is constant.)

Implementing a stack from scratch

Array-based (a Python list is a dynamic array):

```
class ArrayStack:
    """A stack on top of a Python list (dynamic array): push/pop at the END are  $O(1)$  amortized."""

    def __init__(self):
        self._data = []

    def push(self, x):
        self._data.append(x)

    def pop(self):
        if not self._data:
            raise IndexError("pop from empty stack")      # stack UNDERFLOW
        return self._data.pop()

    def peek(self):
        if not self._data:
            raise IndexError("peek from empty stack")
        return self._data[-1]

    def is_empty(self):
        return not self._data

    def __len__(self):
        return len(self._data)
```

Linked-list-based — the *head* is the top, so push and pop are $O(1)$ **worst case** and there is never any resizing:

```

class LinkedStack:
    """A stack on a singly linked list: the HEAD is the top, so push/pop are O(1) *worst case*."""

    def __init__(self):
        self._head = None
        self._size = 0

    def push(self, x):
        self._head = _Node(x, self._head)    # new node points at the old top
        self._size += 1

    def pop(self):
        if self._head is None:
            raise IndexError("pop from empty stack")
        node = self._head
        self._head = node.next
        self._size -= 1
        return node.val

    def peek(self):
        if self._head is None:
            raise IndexError("peek from empty stack")
        return self._head.val

    def is_empty(self):
        return self._head is None

    def __len__(self):
        return self._size

class _Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.next = nxt

```

	Array (list) stack	Linked stack
push / pop	O(1) amortized / O(1)	O(1) worst case
Memory	compact, cache-friendly	extra pointer per node
Random access	possible (but not allowed by the discipline)	none
Typical use	almost always	when worst-case latency matters

1.7 The core stack mental model

Forget the syntax for a moment. Processing $2 \rightarrow 1 \rightarrow 5 \rightarrow 3 \rightarrow 4$, an ordinary algorithm thinks "*I am processing elements one by one.*" A **stack-based algorithm often thinks: "Which elements have not been resolved yet?"** — that is the huge shift.

Suppose the question is "*for every number, find the next number greater than it.*"

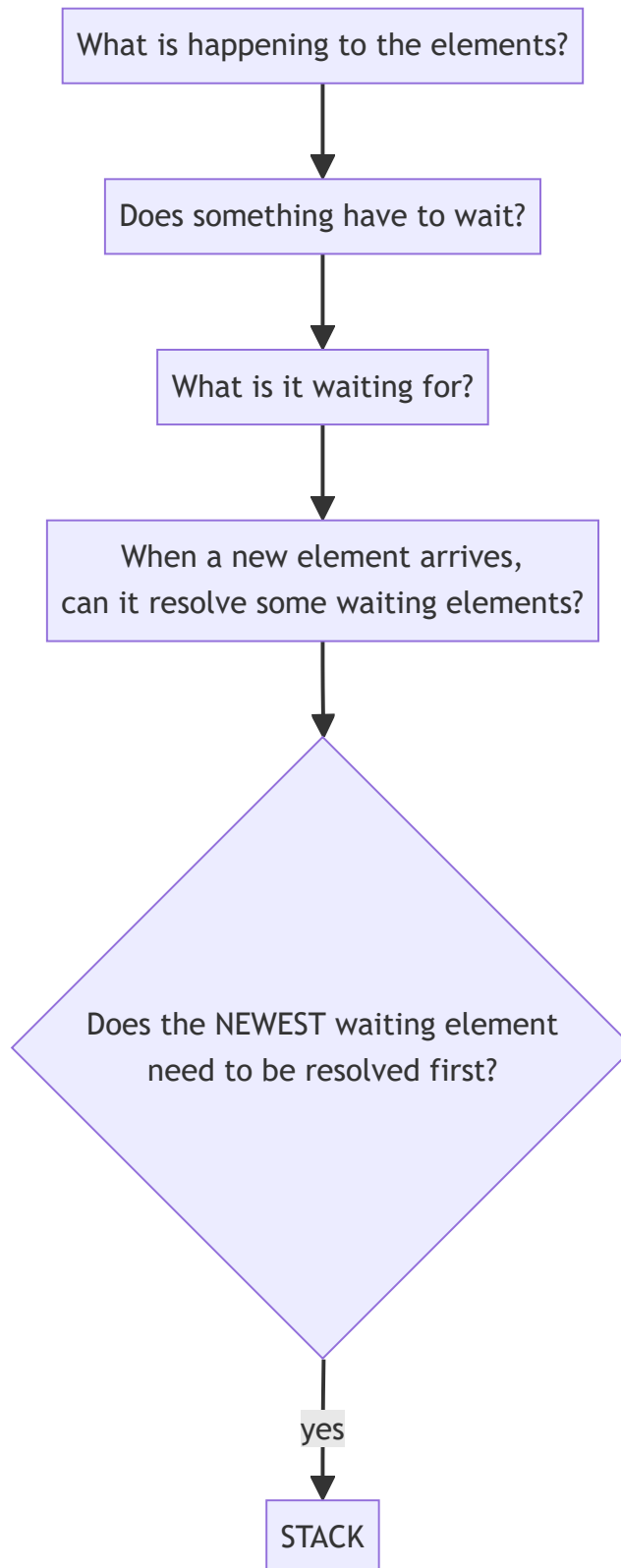
- See $2 \rightarrow$ we don't know its answer yet. **2 is waiting.**
- See $1 \rightarrow$ 1 doesn't resolve 2 . Now 1 is waiting too: waiting: $2, 1$
- See $5 \rightarrow$ $5 > 1$ and $5 > 2$. **Both waiting elements are resolved.**

```
5
↓ ↓
[1, 2] ← waiting
↓
both resolved
```

And notice something subtle: **5 resolves the most recently waiting element first** (1 , then 2). That naturally matches LIFO. This is the seed of the monotonic stack.

1.8 The fundamental recognition idea

When you see a new DSA problem don't ask "*Is this a stack problem?*" Ask:



If you remember one thing from Part 1: a stack is useful when **the most recently encountered unresolved thing needs to be handled before older unresolved things.**

Why not a queue? (from the session) Suppose **A** and **B** are waiting and **C** arrives and can resolve both. With a **stack** the waiting elements are **[A, B]** with **B** on top → **B** is checked first, then **A**. A **queue** would force **A** to be checked first, which does not match the structure. So the useful property is not the textbook "LIFO" — it is that **the most recently waiting element gets the chance to be resolved first.**

1.9 What can go wrong — quick facts

Situation	What happens
<code>pop()</code> / <code>[-1]</code> on an empty list	<code>IndexError</code> — check <code>if stack</code> first
Recursion too deep	<code>RecursionError</code> (Python's call-stack limit is about 1000 frames by default) → convert to an explicit stack
Using <code>pop(0)</code> as "pop"	works but $O(n)$ — bug in disguise
Comparing <code>stack[-1]</code> when the stack stores <i>indices</i>	you compare indices, not values! use <code>nums[stack[-1]]</code>

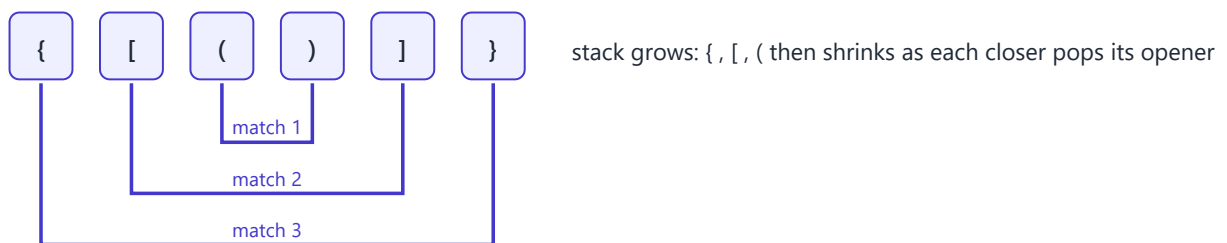
Part 2 · Matching and Cancelling Problems

Recognition. *Nested or paired structure* — an **opener waits for its closer**, and the closer must match the **most recent** unmatched opener. Or: *an item is cancelled by its neighbour*.

2.1 Valid Parentheses (LeetCode 20)

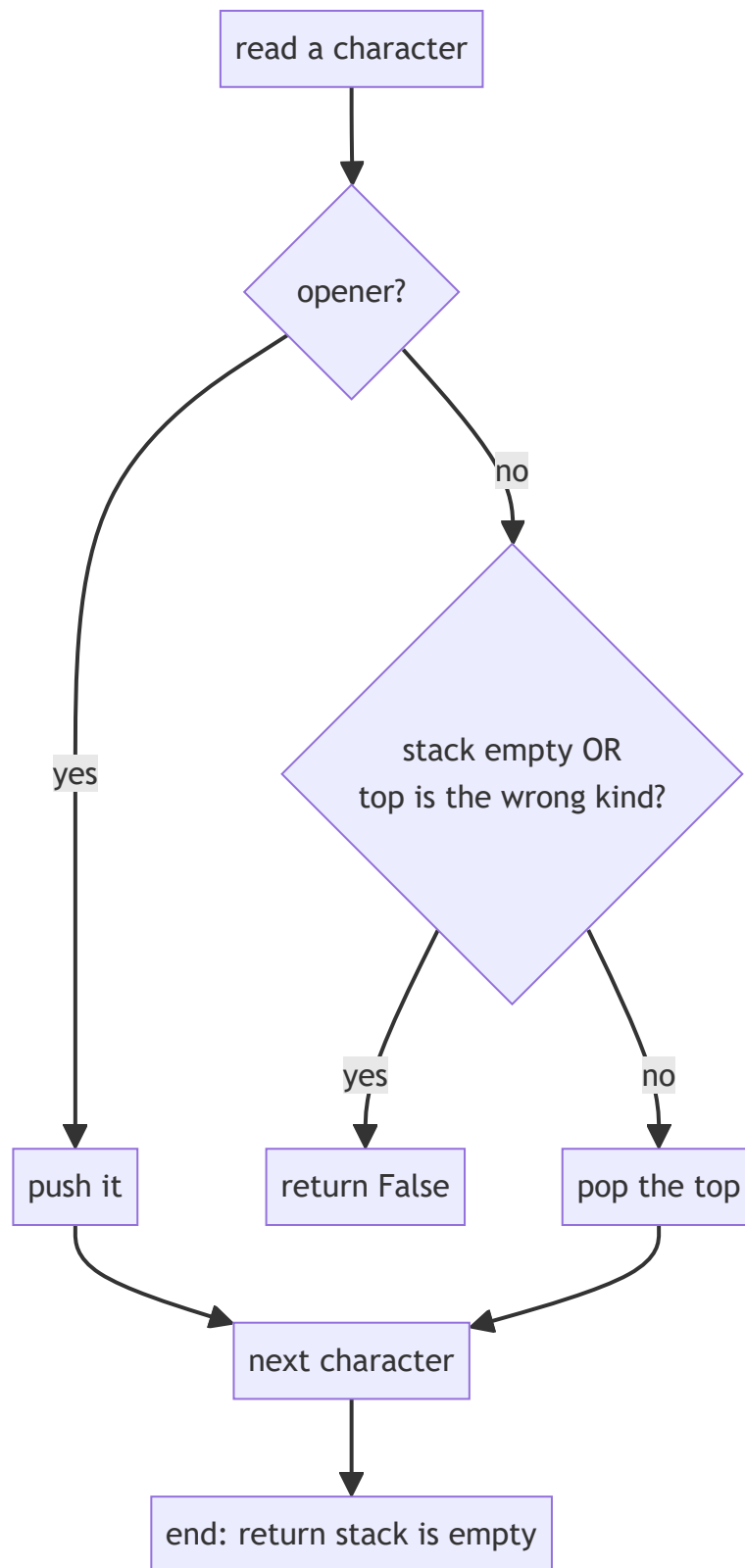
Given a string of `()[]{}` , decide whether every bracket is closed by the right kind, in the right order. `"()[]{}"` → true, `"(]"` → false, `"{[]}"` → true, `"([)]"` → false.

Nested brackets: each closer must match the **MOST RECENT** unmatched opener



Derive it.

- What is each opener waiting for? *Its matching closer*.
- When a closer arrives, which opener must it match? **The most recent unmatched one** — the top of the stack.
- What does the stack contain? *Openers still waiting*.
- Leftovers at the end? *Openers that were never closed* → *invalid*.



char	action	stack (bottom → top)
{	opener: push	[{]
[	opener: push	[{, [
(	opener: push	[{, [, (
)	closer) matches top (: pop	[{, [

char	action	stack (bottom → top)
]	closer] matches top [: pop	[]
}	closer } matches top {: pop	[]

The failing case shows the *wrong kind on top* rule:

char	action	stack (bottom → top)
(	opener: push	[(
[	opener: push	[(, [
)	closer) but top is [: INVALID	[(, [

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in pairs:
            # a closing bracket
            if not stack or stack[-1] != pairs[ch]:
                # nothing to match, or the wrong kind on
                return False
            stack.pop()
        else:
            # an opening bracket
            stack.append(ch)
    return not stack
    # leftovers = unmatched openers
```

Complexity. Each character is pushed at most once and popped at most once → **O(n)** time, **O(n)** space. (For a *single* kind of bracket a plain counter is enough — the stack is needed only when several kinds must nest correctly.)

2.2 Minimum Add to Make Parentheses Valid (LeetCode 921)

"())" → 1, "((((" → 3, "()))((" → 4. Only (and).

Idea. With one bracket kind the stack degenerates into a **counter** of unmatched (. A) with no (to match needs an added (; leftover (need added).

```
def min_add_to_make_valid(s):
    open_needed = 0      # unmatched '(' waiting on the stack (only one kind, so a counter is enough)
    add = 0              # unmatched ')' -> each needs a '(' added
    for ch in s:
        if ch == '(':
            open_needed += 1
        elif open_needed:
            open_needed -= 1
        else:
            add += 1
    return add + open_needed
```

2.3 Minimum Remove to Make Valid Parentheses (LeetCode 1249)

Remove the fewest (/) so the string is valid: "lee(t(c)o)de)" → "lee(t(c)o)de" .

Idea. Push the **indices** of (. A) pops if there is a partner, otherwise that) is invalid → mark it for deletion. At the end, every index still on the stack is an unmatched (→ mark those too. **Storing indices (not characters) is what lets us delete by position.**

```
def min_remove_to_make_valid(s):
    chars = list(s)
    stack = []                                # indices of '(' still unmatched
    for i, ch in enumerate(chars):
        if ch == '(':
            stack.append(i)
        elif ch == ')':
            if stack:
                stack.pop()
            else:
                chars[i] = ''                # ')' with no partner: drop it
    for i in stack:
        chars[i] = ''                        # '(' that never got closed: drop them
    return ''.join(chars)
```

2.4 Longest Valid Parentheses (LeetCode 32)

"()(())" → 4 (the substring "()()").

Idea. Keep a stack of **indices**, seeded with a base `-1` . The top of the stack is always "the index just before the current valid run begins".

- (→ push its index.

- `)` → pop. If the stack is now empty this `)` is unmatched → push its index as the **new base**. Otherwise the current valid run has length `i - stack[-1]`.

i: char	action	stack of indices	best
0:)	no base left: this ')' is the new base	[0]	0
1: (	push index	[0, 1]	0
2:)	valid run length = 2 - 0 = 2	[0]	2
3: (	push index	[0, 3]	2
4:)	valid run length = 4 - 0 = 4	[0]	4
5:)	no base left: this ')' is the new base	[5]	4

Longest valid substring = **4**

```
def longest_valid_parentheses(s):
    stack = [-1]                                # base index = "the position just before
    the current run"
    best = 0
    for i, ch in enumerate(s):
        if ch == '(':
            stack.append(i)
        else:
            stack.pop()
            if not stack:
                stack.append(i)                  # unmatched ')': it becomes the new base
            else:
                best = max(best, i - stack[-1])  # length of the valid run ending at i
    return best
```

Complexity. $O(n)$ time, $O(n)$ space.

2.5 Score of Parentheses (LeetCode 856)

`"()"` = 1, `"AB"` = A + B, `"(A)"` = 2 × A. `"(()())"` → 6.

Idea. The stack holds the **score accumulated at each open depth**. A `(` opens a new depth (push `0`); a `)` closes it: `inner = pop()`, and the parent gains `max(2 × inner, 1)` (an empty `()` scores 1).

```
def score_of_parentheses(s):
    stack = [0]                                # stack[-1] = score accumulated at the
    current depth
    for ch in s:
        if ch == '(':
            stack.append(0)                    # open a new depth
        else:
            inner = stack.pop()
            stack[-1] += max(2 * inner, 1)      # "()" = 1, "(A)" = 2 * A
    return stack[0]
```

2.6 Reverse Substrings Between Each Pair of Parentheses (LeetCode 1190)

"(u(love)i)" → "iloveu".

Idea. Each `(` starts a new "current text" and saves the old one on the stack. Each `)` reverses the current text and appends it to the saved text.

```
def reverse_parentheses(s):
    stack = []                                # stack of partially built strings
    cur = []
    for ch in s:
        if ch == '(':
            stack.append(cur)
            cur = []
        elif ch == ')':
            cur.reverse()                      # reverse what was inside the brackets
            prev = stack.pop()
            prev.extend(cur)
            cur = prev
        else:
            cur.append(ch)
    return ''.join(cur)
```

2.7 Valid Parenthesis String (LeetCode 678)

`*` may be `(`, `)` or empty. `"(*)")" → true`.

Idea. Two stacks of **indices** — one for `(`, one for `*`. A `)` first uses a `(`, else a `*`. At the end match leftover `(` with `*` that come **after** them (a `*` acting as `)` must be to the right).

```
def check_valid_string(s):
    """'*' may be '(' , ')' or empty. Two stacks of indices."""
    opens, stars = [], []
    for i, ch in enumerate(s):
        if ch == '(':
            opens.append(i)
        elif ch == '*':
            stars.append(i)
        else:
            if opens:
                opens.pop()
            elif stars:
                stars.pop()
            else:
                return False
    while opens and stars:
        if opens.pop() > stars.pop():
            # a '*' must come AFTER the '(' it closes
            return False
    return not opens
```

2.8 Cancelling problems

Remove All Adjacent Duplicates in String (LeetCode 1047)

"abbaca" → "ca" (remove bb, then aa).

The **top of the stack is the previous surviving character**; if the new character equals it, they cancel. Chain reactions (aa exposing bb) are handled automatically because after a pop the new top is compared next time.

```
def remove_duplicates(s):
    stack = []
    for ch in s:
        if stack and stack[-1] == ch:
            stack.pop()
            # the pair cancels out
        else:
            stack.append(ch)
    return ''.join(stack)
```

Backspace String Compare (LeetCode 844)

s = "ab#c", t = "ad#c" → both become "ac" → true.

Simulate typing with a stack: a letter is pushed, a # pops (if anything is there). Compare the two final stacks. (An O(1) -space two-pointer solution exists by scanning from the right, but the stack version is the clearest.)

```
def backspace_compare(s, t):
    def build(text):
        stack = []
        for ch in text:
            if ch == '#':
                if stack:
                    stack.pop()
            else:
                stack.append(ch)
        return stack
    return build(s) == build(t)
```

Asteroid Collision (LeetCode 735)

Positive = moving right, negative = moving left, same speed; the smaller explodes, equal both explode. $[5, 10, -5] \rightarrow [5, 10]$; $[8, -8] \rightarrow []$; $[10, 2, -5] \rightarrow [10]$.

Derive it. What is waiting? Right-moving asteroids (they *might* be hit later). What resolves them? A left-moving asteroid arriving. Which is hit first? **The nearest one — the top of the stack.** A collision only happens when the *top* is *positive* and the *newcomer* is *negative*.

```
newcomer < 0 and top > 0:
    top < |newcomer| → top explodes, keep checking the next top
    top == |newcomer| → both explode
    top > |newcomer| → the newcomer explodes
otherwise push the newcomer
```

asteroid	collisions	stack
5	no collision	[5]
10	no collision	[5, 10]
-5	-5 explodes	[5, 10]

asteroid	collisions	stack
8	no collision	[8]
-8	8 and -8 both explode	[]
3	no collision	[3]
-2	-2 explodes	[3]
-5	3 explodes	[-5]

```

def asteroid_collision(asteroids):
    stack = []
    for a in asteroids:
        alive = True
        while alive and a < 0 and stack and stack[-1] > 0:      # right-mover on top meets a left-
mover
            if stack[-1] < -a:
                stack.pop()                                     # the top explodes, keep checking
            elif stack[-1] == -a:
                stack.pop()
                alive = False                                   # both explode
            else:
                alive = False                                   # the newcomer explodes
        if alive:
            stack.append(a)
    return stack

```

Complexity. Each asteroid is pushed once and popped at most once → **O(n)**.

Part 3 · Expression Evaluation and Parsing

Recognition. Operands and operators must be combined in an order that depends on precedence or nesting → keep the pending pieces on a stack.

3.1 Evaluate Reverse Polish Notation (LeetCode 150)

```
["2","1","+","3","*"] → ((2 + 1) * 3) = 9 .
```

In **postfix (RPN)** notation the operator comes *after* its operands, so no parentheses or precedence rules are needed: *push operands; on an operator pop the two most recent operands, apply, push the result.*

token	action	stack
2	operand: push	[2]
1	operand: push	[2, 1]
+	pop 1, pop 2 → 2 + 1 = 3	[3]
3	operand: push	[3, 3]
*	pop 3, pop 3 → 3 * 3 = 9	[9]

Result = 9

The classic slip: the **first** value you pop is the **right** operand `b`, the second is the left operand `a`. For `-` and `/` the order matters: `a - b`, `a / b`. Division truncates toward zero (`int(a / b)`).

```
def eval_rpn(tokens):
    stack = []
    for t in tokens:
        if t in ('+', '-', '*', '/'):
            b = stack.pop()                # right operand FIRST (it was pushed
last)
            a = stack.pop()
            if t == '+':
                stack.append(a + b)
            elif t == '-':
                stack.append(a - b)
            elif t == '*':
                stack.append(a * b)
            else:
                stack.append(int(a / b))    # truncate toward zero
        else:
            stack.append(int(t))
    return stack[0]
```

Complexity. $O(n)$ time and space.

3.2 Basic Calculator II (LeetCode 227)

`+` `-` `*` `/` on non-negative integers, no parentheses: `"3+2*2"` $\rightarrow 7$, `" 3/2 "` $\rightarrow 1$, `" 3+5 / 2 "` $\rightarrow 5$.

Idea. Precedence: `*` and `/` bind tighter, so they must be applied **immediately** to the previous term, while `+` `-` terms are just collected. The stack holds **the signed terms**; at the end the answer is `sum(stack)`.

The trick that confuses everyone: when you *finish reading a number* you apply the **operator that came before it** (`op`), not the one you just met.

char	event	action (uses the PREVIOUS operator)	stack of terms
'+' (i=1)	number 3 finished	op '+': push 3	[3]
'*' (i=3)	number 2 finished	op '+': push 2	[3, 2]
'2' (i=4)	number 2 finished	op '*': $\text{top} \times 2 = 4$	[3, 4]

Answer = sum of the stack = **7**

```
def calculate_ii(s):
    """'3+2*2' -> 7 ; + - * / on non-negative integers, no parentheses."""
    stack = []
    num = 0
    op = '+' # the operator that came BEFORE `num`

    for i, ch in enumerate(s):
        if ch.isdigit():
            num = num * 10 + int(ch)
        if ch in '+-*/' or i == len(s) - 1:
            if op == '+':
                stack.append(num)
            elif op == '-':
                stack.append(-num)
            elif op == '*':
                stack.append(stack.pop() * num) # * and / bind tighter: fold into
            elif op == '/':
                stack.append(int(stack.pop() / num))
            op = ch
            num = 0
    return sum(stack)
```

Complexity. $O(n)$ time, $O(n)$ space. (The stack can be replaced by a `last_term` variable for $O(1)$ space.)

3.3 Basic Calculator (LeetCode 224)

`+` `-` `( )`, unary minus allowed: `"(1+(4+5+2)-3)+(6+8)"` $\rightarrow 23$, `"- (3 + (4 + 5))"` $\rightarrow -12$.

Idea. There is no precedence problem, but **parentheses nest**. Keep the running `result` and the current `sign`. On `(` **save (result, sign) on the stack and start fresh**; on `)` finish the inner value, multiply by the saved sign, and add the saved result.

```
def calculate(s):
    """Basic Calculator: + - ( ) and unary minus."""
    stack = []
    # saves (result_so_far, sign) when a
    '(' opens
    result, num, sign = 0, 0, 1
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch in '+-':
            result += sign * num
            num = 0
            sign = 1 if ch == '+' else -1
        elif ch == '(':
            stack.append(result)
            stack.append(sign)
            result, sign = 0, 1
            # start fresh inside the brackets
        elif ch == ')':
            result += sign * num
            num = 0
            result *= stack.pop()
            result += stack.pop()
            # the sign in front of '('
            # what we had before '('
    return result + sign * num
```

3.4 Infix → Postfix (Shunting-Yard) — the general method

For expressions with several precedence levels and parentheses, convert infix to postfix using an **operator stack**, then evaluate with §3.1.

- **Operand** → output.
- **Operator** `o` → while the stack top is an operator with **higher or equal precedence** (left-associative), pop it to the output; then push `o`.
- `(` → push. `)` → pop to the output until `(`, discard the `(`.
- End → pop everything.

`3 + 4 * 2` → `3 4 2 * +`

```
def infix_to_postfix(tokens):
    """Shunting-yard for + - * / and parentheses. tokens is a list like ['3','+', '4','*', '2']."""
    prec = {'+': 1, '-': 1, '*': 2, '/': 2}
    out, ops = [], []
    for t in tokens:
        if t in prec:
            while ops and ops[-1] != '(' and prec[ops[-1]] >= prec[t]: # left-associative
                out.append(ops.pop())
            ops.append(t)
        elif t == '(':
            ops.append(t)
        elif t == ')':
            while ops[-1] != '(':
                out.append(ops.pop())
            ops.pop() # discard the '('
        else:
            out.append(t)
    while ops:
        out.append(ops.pop())
    return out
```

3.5 Decode String (LeetCode 394)

"3[a2[c]]" → "accaccacc", "2[abc]3[cd]ef" → "abccabccdcddcdef".

Derive it. Brackets nest, so on `[` we must *suspend* what we were building and remember **how many times** to repeat the inner text. That pair (text before, repeat count) goes on the stack; on `]` we resume: `cur = text_before + cur × count`.

char	action	cur	stack of (text before, count)
3	digit: build the repeat count	"	[]
[	'[': save ('', 3) and start fresh	"	[('',3)]
a	letter: append to cur	'a'	[('',3)]
2	digit: build the repeat count	'a'	[('',3)]
[	'[': save ('a', 2) and start fresh	"	[('',3), ('a',2)]
c	letter: append to cur	'c'	[('',3), ('a',2)]
]	']': cur = 'a' + cur × 2	'acc'	[('',3)]
]	']': cur = '' + cur × 3	'accaccacc'	[]

Result = **accaccacc**

```
def decode_string(s):
    stack = [] # saves (text_before, repeat_count)
    cur, k = '', 0
    for ch in s:
        if ch.isdigit():
            k = k * 10 + int(ch)
        elif ch == '[':
            stack.append((cur, k))
            cur, k = '', 0
        elif ch == ']':
            prev, times = stack.pop()
            cur = prev + cur * times
        else:
            cur += ch
    return cur
```

Complexity. $O(\text{length of the output})$.

3.6 Simplify Path (LeetCode 71)

`"/a/./b/../../c/"` → `"/c"`.

Split on `/`. `"` and `.` are ignored, `..` **pops** (go up a directory), anything else is **pushed**. Join with `/`.

component	action	stack
<code>"</code>	ignore	<code>[]</code>
<code>'a'</code>	push directory	<code>[a]</code>
<code>'.'</code>	ignore	<code>[a]</code>
<code>'b'</code>	push directory	<code>[a, b]</code>
<code>'..'</code>	pop (go up)	<code>[a]</code>
<code>'..'</code>	pop (go up)	<code>[]</code>
<code>'c'</code>	push directory	<code>[c]</code>
<code>"</code>	ignore	<code>[c]</code>

Canonical path = `/c`

```
def simplify_path(path):
    stack = []
    for part in path.split('/'):
        if part in ('', '.'):
            continue
        if part == '..':
            if stack:
                stack.pop()
        else:
            stack.append(part)
    return '/' + '/'.join(stack)
```

3.7 Exclusive Time of Functions (LeetCode 636)

Logs "0:start:0", "1:start:2", "1:end:5", "0:end:6" → [3, 4].

This is the **call stack itself**: on `start` the function on top pauses (charge it the time it just ran) and the new function is pushed; on `end` the top finishes (charge its time; the `end` timestamp is *inclusive*, so the next running function resumes at `ts + 1`).

log	meaning	call stack	exclusive time so far
0:start:0	function 0 starts; caller ran until now	[0]	[0, 0]
1:start:2	function 1 starts; caller ran until now	[0, 1]	[2, 0]
1:end:5	function 1 ends (inclusive)	[0]	[2, 4]
0:end:6	function 0 ends (inclusive)	[]	[3, 4]

```
def exclusive_time(n, logs):
    """logs like ['0:start:0', '1:start:2', '1:end:5', '0:end:6'] -> exclusive time per function."""
    result = [0] * n
    stack = [] # function ids currently on the call stack
    prev = 0
    for log in logs:
        fid, kind, ts = log.split(':')
        fid, ts = int(fid), int(ts)
        if kind == 'start':
            if stack:
                result[stack[-1]] += ts - prev # the caller ran until now
            stack.append(fid)
            prev = ts
        else:
            result[stack.pop()] += ts - prev + 1 # end timestamp is inclusive
            prev = ts + 1
    return result
```

Part 4 · Design Problems with Stacks

Recognition. "Support push / pop and also get the minimum / maximum / do X in $O(1)$ " → store **extra information with every stack element**.

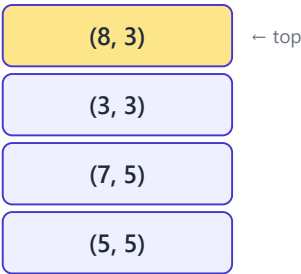
4.1 Min Stack (LeetCode 155)

Design a stack with push , pop , top and get_min , all in **$O(1)$** .

Derive it. The minimum can *change* when we pop (the old minimum leaves). But the minimum **of the stack beneath the top never changes while that entry stays** — so store, with every entry, *the minimum at the time it was pushed*.

Min Stack: every entry remembers the minimum at the time it was pushed

(value, min so far)



get_min() = top.min = **3** ($O(1)$)

pop() removes (8, 3): new top (3, 3) -> min is still 3

pop() again removes (3, 3): top (7, 5) -> min is back to 5

No search needed: the old minimum is restored automatically because it was stored with the entry underneath.

operation	returns	stack of (value, min so far)
push(5)		[(5, 5)]
push(7)		[(5, 5), (7, 5)]
push(3)		[(5, 5), (7, 5), (3, 3)]
push(8)		[(5, 5), (7, 5), (3, 3), (8, 3)]
get_min()	→ 3	[(5, 5), (7, 5), (3, 3), (8, 3)]
pop()	→ 8	[(5, 5), (7, 5), (3, 3)]
pop()	→ 3	[(5, 5), (7, 5)]
get_min()	→ 5	[(5, 5), (7, 5)]

```

class MinStack:
    """push/pop/top/get_min all in O(1). Each entry remembers the minimum at the time it was
    pushed."""

    def __init__(self):
        self.stack = [] # (value, min_so_far)

    def push(self, x):
        cur_min = x if not self.stack else min(x, self.stack[-1][1])
        self.stack.append((x, cur_min))

    def pop(self):
        return self.stack.pop()[0]

    def top(self):
        return self.stack[-1][0]

    def get_min(self):
        return self.stack[-1][1]

```

The same idea gives a **Max Stack**:

```

class MaxStack:
    """Same idea for the maximum (peek_max in O(1))."""

    def __init__(self):
        self.stack = []

    def push(self, x):
        cur = x if not self.stack else max(x, self.stack[-1][1])
        self.stack.append((x, cur))

    def pop(self):
        return self.stack.pop()[0]

    def top(self):
        return self.stack[-1][0]

    def peek_max(self):
        return self.stack[-1][1]

```

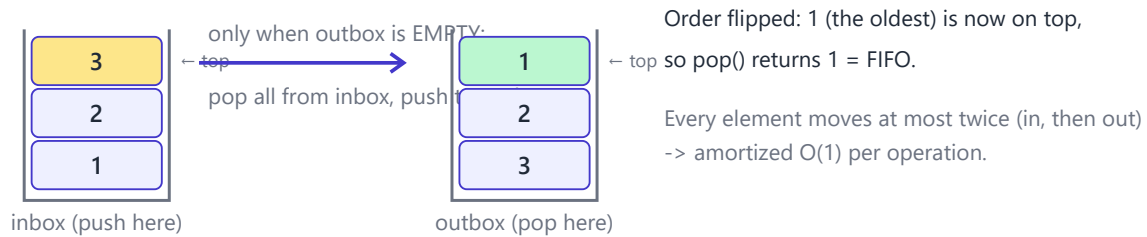
Alternative: keep a second stack of minimums, pushing only when the new value is $\leq$ the current minimum. Same $O(1)$ operations; the pair-per-entry version is simpler to get right.

4.2 Implement a Queue Using Stacks (LeetCode 232)

FIFO from two LIFO stacks.

Derive it. Popping all of stack A into stack B **reverses** the order, so the *oldest* element ends up on top of B. So: push into an **inbox**; pop from an **outbox**; **only when the outbox is empty**, refill it by moving *everything* from the inbox.

Queue from two stacks: inbox for push, outbox for pop



operation	returns	inbox (bottom→top)	outbox (bottom→top)
push(1)		[1]	[]
push(2)		[1, 2]	[]
push(3)		[1, 2, 3]	[]
pop()	→ 1	[]	[3, 2]
push(4)		[4]	[3, 2]
pop()	→ 2	[4]	[3]
pop()	→ 3	[4]	[]
pop()	→ 4	[]	[]

```

class QueueUsingStacks:
    """FIFO queue from two LIFO stacks: amortized O(1) per operation."""

    def __init__(self):
        self.inbox = []           # new items are pushed here
        self.outbox = []         # items are popped from here (oldest
on top)

    def push(self, x):
        self.inbox.append(x)

    def _move(self):
        if not self.outbox:      # only refill when the outbox is
EMPTY
            while self.inbox:
                self.outbox.append(self.inbox.pop())    # reverses the order: oldest ends on
top

    def pop(self):
        self._move()
        return self.outbox.pop()

    def peek(self):
        self._move()
        return self.outbox[-1]

    def empty(self):
        return not self.inbox and not self.outbox

```

Complexity. Each element is moved at most once from inbox to outbox → **amortized O(1)** per operation (the occasional transfer is paid for by the pushes that put those elements there).

4.3 Implement a Stack Using Queues (LeetCode 225)

Idea. After every `push`, **rotate** the queue so the newest element sits at the front: push at the back, then move the other `n - 1` elements behind it. Then `pop / top` are just `popLeft / front`. (`push` is $O(n)$, `pop` $O(1)$.)

```

class StackUsingQueue:
    """LIFO stack from ONE queue: after each push, rotate the queue so the newest item is at the
    front."""

    def __init__(self):
        self.q = deque()

    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())           # rotate the older items behind x

    def pop(self):
        return self.q.popleft()

    def top(self):
        return self.q[0]

    def empty(self):
        return not self.q

```

4.4 Design Browser History (LeetCode 1472)

Idea. Two stacks: **back** (last element = current page) and **forward**. Going back moves the current page to **forward**; visiting a new URL clears **forward**.

```

class BrowserHistory:
    """Back / forward with two stacks."""

    def __init__(self, homepage):
        self.back = [homepage]                # last element = current page
        self.forward = []

    def visit(self, url):
        self.back.append(url)
        self.forward.clear()                  # visiting a new page kills the
forward history

    def go_back(self, steps):
        while steps and len(self.back) > 1:
            self.forward.append(self.back.pop())
            steps -= 1
        return self.back[-1]

    def go_forward(self, steps):
        while steps and self.forward:
            self.back.append(self.forward.pop())
            steps -= 1
        return self.back[-1]

```

4.5 Design a Stack With Increment Operation (LeetCode 1381)

`increment(k, val)` adds `val` to the bottom `k` elements. Make everything $O(1)$.

Lazy trick. Don't touch `k` elements; just record `inc[k-1] += val`. When the top is popped, **pass its pending increment down** to the element beneath it, and add it to the value being returned.

```

class CustomStack:
    """Stack with increment(k, val): add val to the bottom k elements. Lazy trick: O(1)
    everything."""

    def __init__(self, max_size):
        self.max_size = max_size
        self.stack = []
        self.inc = []  # inc[i] = pending increment for
                        # stack[0..i]

    def push(self, x):
        if len(self.stack) < self.max_size:
            self.stack.append(x)
            self.inc.append(0)

    def pop(self):
        if not self.stack:
            return -1
        add = self.inc.pop()
        if self.inc:
            self.inc[-1] += add  # pass the pending increment down
        return self.stack.pop() + add

    def increment(self, k, val):
        k = min(k, len(self.stack))
        if k:
            self.inc[k - 1] += val  # remember it at the k-th element
only

```

4.6 Validate Stack Sequences (LeetCode 946)

Given `pushed` and `popped`, could they be the results of a valid sequence of pushes and pops on an initially empty stack? `pushed=[1,2,3,4,5]`, `popped=[4,5,3,2,1]` → true.

Idea. Simulate: push each element, and whenever the top equals the next expected pop, pop **greedily**. It is valid iff the stack is empty at the end.

```
def validate_stack_sequences(pushed, popped):
    stack = []
    j = 0
    for x in pushed:
        stack.append(x)
        while stack and j < len(popped) and stack[-1] == popped[j]:
            stack.pop()
            # the expected pop is available: do
    it now
    j += 1
    return not stack
```

4.7 Baseball Game (LeetCode 682)

["5", "2", "C", "D", "+"] → 30. **c** cancels the previous score, **D** doubles it, **+** adds the previous two.

A direct stack simulation:

```
def cal_points(operations):
    stack = []
    for op in operations:
        if op == '+':
            stack.append(stack[-1] + stack[-2])
        elif op == 'D':
            stack.append(2 * stack[-1])
        elif op == 'C':
            stack.pop()
        else:
            stack.append(int(op))
    return sum(stack)
```

Part 5 · The Monotonic Stack

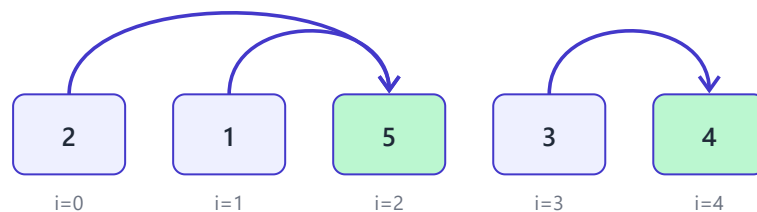
Recognition. "For each element find the nearest greater / smaller element to its left or right" — or any problem that secretly needs it (waiting days, spans, next bigger, widest range where a value is the min/max). This is probably **the most important stack pattern for interviews**.

5.1 Deriving it: Next Greater Element

For `[2, 1, 5, 3, 4]`, for each element find the first element to its **right** that is **greater** than it.

`2 → 5 1 → 5 5 → ? 3 → 4 4 → ? answer = [5, 5, -1, 4, -1]`

Next Greater Element: every element points to the first larger value on its right



`answer = [5, 5, -1, 4, -1]` (green = no greater element to the right)

2 and 1 both wait for something bigger; 5 arrives and resolves BOTH (newest waiting first)

Step 1 — what does each element wait for? An element that is **greater than it**. 2 and 1 have not found theirs yet.

Step 2 — why can 5 resolve both 1 and 2? The precise reason (this matters for monotonic stacks): it is not merely "5 is greater than both" — **5 is greater than the newest waiting element (1), and after resolving it, 5 is also able to resolve the older waiting element (2)**. That is exactly where LIFO helps.

Step 3 — derive the stack. Process left → right:

2 arrives → nothing to resolve → 2 waits	stack: [2]
1 arrives → <code>1 < 2</code> , cannot resolve 2 → 1 waits	stack: [2, 1] (1 is the newest waiting)
5 arrives → <code>5 > 1</code> → resolve 1; <code>5 > 2</code> → resolve 2	stack: [] → push 5 → [5]

The stack contains elements that are still waiting for their answer.

Step 4 — what condition allows a pop? Stack `[2, 1]` (top 1), current 3. Pop 1? Pop 2? Push without popping?

Pop both, because 3 is greater than both. We didn't memorise `while stack and stack[-1] < current: pop()` — we derived it:

If the current element can answer the question of the element on top of the stack, pop that element. For Next Greater "can answer" means `current > stack_top`.

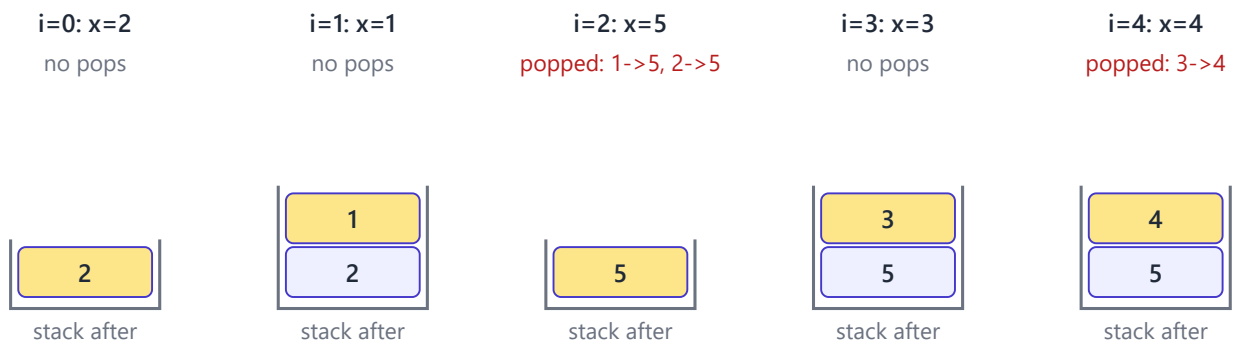
Step 5 — what does the stack look like? After the whole array the stack is `[5, 4]`. Values from bottom → top are **decreasing**, so the invariant is:

The elements still waiting for a greater element are kept in decreasing order. That is why it is called a **monotonic decreasing stack**.

Check yourself. Stack [5, 3, 2] (top 2), current 4. Which pop, which stay, what is the final stack?

```
4 > 2 → pop 2
4 > 3 → pop 3
4 < 5 → STOP (5 is still waiting for something greater than 5; 4 cannot answer it)
push 4 → stack [5, 4]
```

Monotonic decreasing stack on [2, 1, 5, 3, 4] (stack holds INDICES; values shown)



Values inside the stack always decrease from bottom to top. What is left at the end (5, 4) never found a greater element -> -1.

5.2 Why is it enough to look only at the top?

(A question from the session, worth answering properly.) It is **not** merely because "the answer is to the right". The real reason is the **invariant**: the stack is already ordered $5 > 3 > 2$. Once $4 < 5$, we know that 4 cannot resolve 5, and every element below 5 is even larger — so nothing deeper can be resolved either. We can stop.

The monotonic stack gives us two things at once:

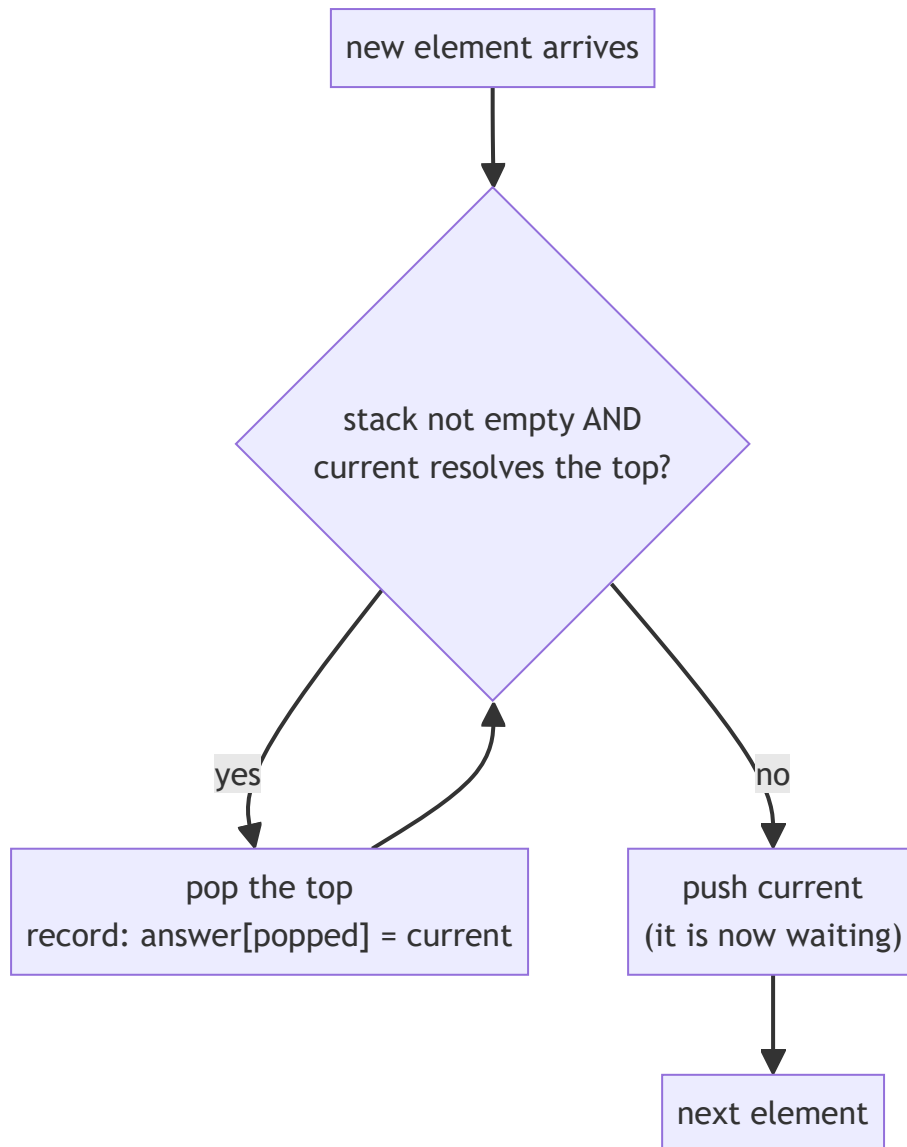
1. **the top** tells us the next unresolved candidate to check;
2. **the ordering** tells us when we can safely stop.

That is what makes the algorithm **O(n)** instead of repeatedly searching through previous elements.

5.3 The complete algorithm in words

For each current element:

1. Look at the top of the stack.
2. If the current element **can resolve** the top: pop it, record that the popped element is resolved *by current*, and repeat.
3. Otherwise stop popping.
4. **Push the current element** — it is itself now an unresolved element waiting for a future greater element.



5.4 Store indices, not just values

The stack alone (`while stack and x > stack[-1]: stack.pop()`) maintains the waiting elements but **doesn't record any answers**. When we pop something we must know *which array position* it belongs to:

```
nums = [2, 1, 5]      when 5 (index 2) arrives:
stack = [0, 1]        nums[1] = 1 → answer[1] = 5
                     nums[0] = 2 → answer[0] = 5
```

The stack stores indices, but we compare the values at those indices: `stack[-1]` is an index, `nums[stack[-1]]` is its value.

5.5 What do the leftovers mean?

After the loop, **the elements still in the stack never found a greater element to their right**. They are not "ignored" — they have a known conclusion: *there is no greater element on their right* → `-1`. Initialising `answer = [-1] * n` handles that automatically.

5.6 The template and the trace

```
def next_greater(nums):
    n = len(nums)
    answer = [-1] * n
    stack = []                                # indices; values DEcrease bottom ->
    top
    for i in range(n):
        while stack and nums[i] > nums[stack[-1]]:
            j = stack.pop()                  # j has just found its next greater
        element
        answer[j] = nums[i]
        stack.append(i)
    return answer                            # whatever is left in the stack
keeps -1
```

i: current	popped (resolved)	stack values after push	answer so far
0: 2	-	[2]	[-1, -1, -1, -1, -1]
1: 1	-	[2, 1]	[-1, -1, -1, -1, -1]
2: 5	1→5, 2→5	[5]	[5, 5, -1, -1, -1]
3: 3	-	[5, 3]	[5, 5, -1, -1, -1]
4: 4	3→4	[5, 4]	[5, 5, -1, 4, -1]

Leftovers `[5, 4]` never found a greater element → they keep `-1`. Final answer: **[5, 5, -1, 4, -1]**

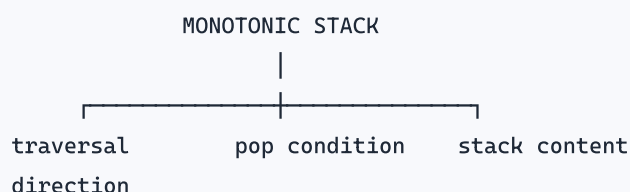
5.7 Generalising: the universal thought process

Whenever you see a possible stack problem, ask these questions **in this order** (derive, don't memorise):

#	Question	Next Greater
1	What is each element waiting for ?	a greater element
2	When does the current element resolve the waiting element?	<code>current > waiting</code> → this is your pop condition
3	Which waiting element is checked first ?	the most recently unresolved → a stack
4	What does the stack maintain? (the invariant)	values decrease bottom → top
5	What does a popped element mean?	<code>nums[j]</code> just found its answer; the answer is <code>nums[i]</code>
6	What should the stack store ?	indices (we need positions for the answer array)

#	Question	Next Greater
7	What do the leftovers mean?	no greater element to the right → <code>-1</code>

The skeleton has only **three** things to determine:



```

stack = []
for i in range(len(nums)):
    while stack and can_resolve(stack[-1], i):
        j = stack.pop()
        answer[j] = ...
    stack.append(i)
  
```

5.8 The four variants — one table

Change **just one relationship** and you get another problem. *Greater/smaller determines the comparison; next/previous determines the direction.*

Problem	Looking	Resolve / pop when	Stack values (bottom → top)
Next Greater	right	<code>current > top</code>	decreasing
Next Smaller	right	<code>current < top</code>	increasing
Previous Greater	left	<code>top <= current</code> (top is useless)	decreasing
Previous Smaller	left	<code>top >= current</code> (top is useless)	increasing

Next Smaller — change one comparison

`nums = [4, 2, 7, 1, 5] → [2, 1, 1, -1, -1]`. Now an element waits for something **smaller**, so `current < top` resolves it and the stack is **increasing** (we keep elements waiting for something smaller).

```

def next_smaller(nums):
    n = len(nums)
    answer = [-1] * n
    stack = []
    # indices; values INcrease bottom ->
    top
    for i in range(n):
        while stack and nums[i] < nums[stack[-1]]:
            j = stack.pop()
            answer[j] = nums[i]
        stack.append(i)
    return answer
  
```

i: current	popped (resolved)	stack values after push	answer so far
0: 4	-	[4]	[-1, -1, -1, -1, -1]
1: 2	4→2	[2]	[2, -1, -1, -1, -1]
2: 7	-	[2, 7]	[2, -1, -1, -1, -1]
3: 1	7→1, 2→1	[1]	[2, 1, 1, -1, -1]
4: 5	-	[1, 5]	[2, 1, 1, -1, -1]

Final answer: **[2, 1, 1, -1, -1]**

The trap from the session: it is easy to answer "smaller element to the *left*" when the question says *next* (right). Always ask: **"From whose perspective am I defining previous / next?"** The popped element `nums[j]` is the one whose answer we are finding; `nums[i]` is the element that gives it that answer. When `1` pops `7`, `1` is to the **right** of `7` → *next* smaller of 7. Also note: at the moment we push an element it is **not resolved** — it is *waiting*, and simultaneously a potential resolver of future elements.

Previous Smaller / Previous Greater — "Form B"

For *previous* problems the current element does not *resolve* older ones; it **asks the stack for its answer**. Everything already in the stack is to its left, so we **pop the elements that can never be the answer** and the top that remains *is* the answer.

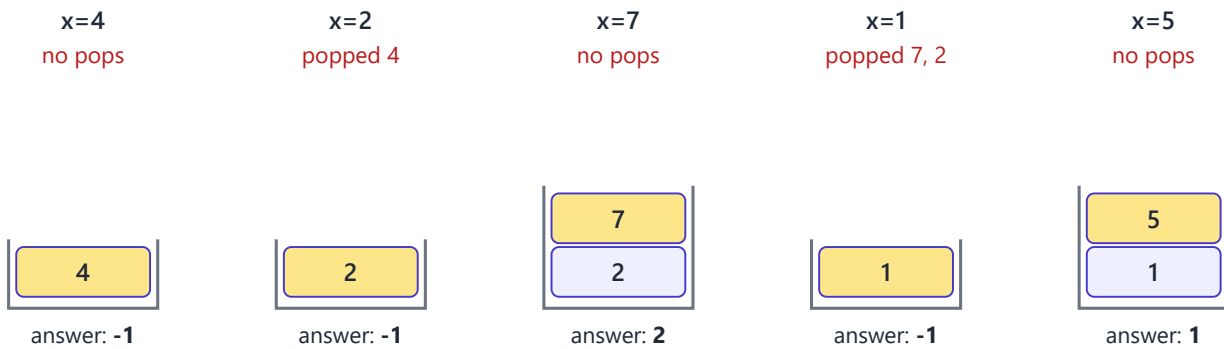
- **Previous Smaller** (`[4, 2, 7, 1, 5]` → `[-1, -1, 2, -1, 1]`): pop while `top >= current` — a top that is `>= current` cannot be *smaller* than current, and cannot be the previous smaller for any *later* element either (current would block it).

```
def previous_smaller(nums):
    n = len(nums)
    answer = [-1] * n
    stack = []
    # values increase bottom -> top
    for i in range(n):
        while stack and nums[stack[-1]] >= nums[i]:
            stack.pop()
        if stack:
            answer[i] = nums[stack[-1]]
        stack.append(i)
    return answer
```

i: current	popped (cannot be the answer)	answer = new top	stack values after push
0: 4	-	-1	[4]
1: 2	4	-1	[2]
2: 7	-	2	[2, 7]
3: 1	7, 2	-1	[1]
4: 5	-	1	[1, 5]

Final answer: **[-1, -1, 2, -1, 1]**

Previous Smaller on [4, 2, 7, 1, 5]: the stack ends up INCREASING (bottom to top)



A [4, 2, 7] stack can never occur: when 2 arrives, 4 ($\geq$ 2) is popped first.

A stack like [4, 2, 7] can never occur for Previous Smaller: when 2 arrives, 4 $\geq$ 2 is popped first. So the real stack for [4, 2, 7, 1, 5] goes [4] $\rightarrow$ [2] $\rightarrow$ [2, 7] $\rightarrow$ [1] $\rightarrow$ [1, 5] — always **increasing** bottom $\rightarrow$ top.

- **Previous Greater** ([4, 2, 7, 1, 5] $\rightarrow$ [-1, 4, -1, 7, 7]): pop while top $\leq$ current.

```
def previous_greater(nums):
    n = len(nums)
    answer = [-1] * n
    stack = []
    for i in range(n):
        while stack and nums[stack[-1]] <= nums[i]:
            stack.pop()
        if stack:
            answer[i] = nums[stack[-1]]
        stack.append(i)
    return answer
```

values decrease bottom $\rightarrow$ top

cannot be the answer for i (or anyone later)

`previous_greater([4, 2, 7, 1, 5]) = [-1, 4, -1, 7, 7]`

The two forms

Form A — the current element *resolves* previous elements (Next Greater / Next Smaller):

```
for i in range(n):
    while stack and condition:
        j = stack.pop()
        answer[j] = nums[i]
    stack.append(i)
```

current is an answer for someone waiting

Form B — the current element *asks* the stack for its answer (Previous Greater / Smaller):

```

for i in range(n):
    while stack and condition:
        stack.pop()          # discard candidates that can't be the answer
    if stack:
        answer[i] = nums[stack[-1]] # current is looking backward
    stack.append(i)

```

Previous X is also solvable with Form A by scanning right → left (the current element then "asks"). Use whichever reads more naturally; the invariants are identical.

5.9 Your universal checklist

1. What is each element waiting for?
2. What event resolves that waiting?
3. Does the newest waiting element get resolved first?
4. What should the stack contain? values / indices?
5. What is the invariant? increasing / decreasing?
6. What condition causes a POP?
7. What does the popped element mean?
8. What does the top mean after popping?
9. What do leftover elements mean?
10. Can each element be pushed once and popped once? → $O(n)$

That last question is the **complexity proof**: every index is pushed exactly once and popped at most once, so the total number of pops $\leq n$ even though there is a `while` loop inside a `for` loop → **$O(n)$ time, $O(n)$ space**.

5.10 Duplicates and strictness (`<` vs `<=`)

"Greater" can mean **strictly** greater or greater-or-equal, and the choice changes the pop rule and the invariant:

Wanted	Pop condition (Form A / B)	Stack is
next strictly greater	pop while <code>nums[i] > top</code>	non-increasing (equal values stay)
next greater-or-equal	pop while <code>nums[i] >= top</code>	strictly decreasing
previous strictly smaller	pop while <code>top >= nums[i]</code>	strictly increasing
previous smaller-or-equal	pop while <code>top > nums[i]</code>	non-decreasing

Correction. The session describes the Next Greater stack as simply "decreasing". With the `>` rule equal values are *not* popped, so the stack is really **non-increasing** (`[3, 3, 2]` is possible). This never changes the algorithm — just don't claim strictness unless you use `>=`.

In problems that count subarrays (Sum of Subarray Minimums, §6.4) you deliberately use `>` **on one side and `>=` on the other** so that ties are attributed to exactly one element.

5.11 Problems built directly on the template

Daily Temperatures (LeetCode 739)

[73, 74, 75, 71, 69, 72, 76, 73] → [1, 1, 4, 2, 1, 1, 0, 0] (days until a warmer day).

Next Greater, but the answer is the **index distance** $i - j$.

day: temp	resolved (days waited)	stack (temp(day)), decreasing
day 0: 73	-	[73(d0)]
day 1: 74	day 0 waited 1	[74(d1)]
day 2: 75	day 1 waited 1	[75(d2)]
day 3: 71	-	[75(d2), 71(d3)]
day 4: 69	-	[75(d2), 71(d3), 69(d4)]
day 5: 72	day 4 waited 1; day 3 waited 2	[75(d2), 72(d5)]
day 6: 76	day 5 waited 1; day 2 waited 4	[76(d6)]
day 7: 73	-	[76(d6), 73(d7)]

Answer: [1, 1, 4, 2, 1, 1, 0, 0]

```
def daily_temperatures(temps):
    n = len(temps)
    ans = [0] * n
    stack = []  # indices of days still waiting for
    a warmer day
    for i, t in enumerate(temps):
        while stack and t > temps[stack[-1]]:
            j = stack.pop()
            ans[j] = i - j  # days waited
        stack.append(i)
    return ans
```

Next Greater Element I (LeetCode 496)

nums1 is a subset of nums2 (unique values). For each x in nums1 find its next greater element in nums2.
nums1=[4,1,2], nums2=[1,3,4,2] → [-1,3,-1].

Run Next Greater over nums2 once, store the answers in a **hash map value → next greater**, then look each nums1 element up.

```
def next_greater_element_i(nums1, nums2):
    """496: for each x in nums1 (a subset of nums2) the next greater element of x inside nums2."""
    nxt = {}
    stack = []
    for x in nums2:
        while stack and x > stack[-1]:
            nxt[stack.pop()] = x
            # values are unique here, so values
            # work as keys
        stack.append(x)
    return [nxt.get(x, -1) for x in nums1]
```

Next Greater Element II — circular array (LeetCode 503)

$[1, 2, 1] \rightarrow [2, -1, 2]$: the last 1 wraps around to find the 2.

Idea. A circular array is scanned **twice**: iterate i from 0 to $2n - 1$ using $\text{nums}[i \% n]$, but **push each index only in the first pass** (so nothing is pushed twice).

position	resolved	stack after
pass 1, i=0: 1	-	[1(i=0)]
pass 1, i=1: 2	1(i=0)→2	[2(i=1)]
pass 1, i=2: 1	-	[2(i=1), 1(i=2)]
pass 2, i=0: 1	-	[2(i=1), 1(i=2)]
pass 2, i=1: 2	1(i=2)→2	[2(i=1)]
pass 2, i=2: 1	-	[2(i=1)]

Answer: **[2, -1, 2]** — the last 1 wraps around and finds 2.

```
def next_greater_circular(nums):
    """503: the array is circular -> scan it twice (2n) but only push each index once."""
    n = len(nums)
    ans = [-1] * n
    stack = []
    for i in range(2 * n):
        x = nums[i % n]
        while stack and x > nums[stack[-1]]:
            ans[stack.pop()] = x
        if i < n:
            stack.append(i)
    return ans
```

Online Stock Span (LeetCode 901)

The span of today's price = number of consecutive days (including today) with price $\leq$ today. Prices 100, 80, 60, 70, 60, 75, 85 $\rightarrow$ spans 1, 1, 1, 2, 1, 4, 6.

This is **Previous Greater with counts**: keep (price, span); today's span = 1 + the spans of every previous entry that is $\leq$ today (they get absorbed, since anything they could see, today sees).

price	popped (price, span) absorbed	span	stack of (price, span)
100	-	1	[(100, 1)]
80	-	1	[(100, 1), (80, 1)]
60	-	1	[(100, 1), (80, 1), (60, 1)]
70	(60,1)	2	[(100, 1), (80, 1), (70, 2)]
60	-	1	[(100, 1), (80, 1), (70, 2), (60, 1)]
75	(60,1), (70,2)	4	[(100, 1), (80, 1), (75, 4)]
85	(75,4), (80,1)	6	[(100, 1), (85, 6)]

```
class StockSpanner:
    """901: span = number of consecutive days (ending today) with price <= today's price."""

    def __init__(self):
        self.stack = []  # (price, span), prices strictly decreasing

    def next(self, price):
        span = 1
        while self.stack and self.stack[-1][0] <= price:
            span += self.stack.pop()[1]  # absorb the span of every smaller-or-equal day
        self.stack.append((price, span))
        return span
```

Next Greater Node in Linked List (LeetCode 1019)

Read the list left to right — it is exactly Next Greater on the values ([2, 1, 5] $\rightarrow$ [5, 5, 0] on LeetCode, where "none" is 0; here -1):

```
def next_greater_nodes(head_vals):
    """1019 on a list of values (the linked list is just read left to right)."""
    return next_greater(head_vals) if head_vals else []
```

Buildings With an Ocean View (LeetCode 1762)

The ocean is on the right; a building has a view if all buildings to its right are strictly shorter. $[4, 2, 3, 1] \rightarrow [0, 2, 3]$.

Idea. A building is blocked as soon as a taller-or-equal one arrives to its right — that is a *pop*. Whatever survives on the stack is unblocked.

```
def buildings_with_ocean_view(heights):
    """1762: the ocean is on the right; return indices of buildings with an unobstructed view."""
    stack = []
    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] <= h:
            stack.pop()
            # blocked by the new, taller-or-
            # equal building
        stack.append(i)
    return stack
```

Maximum Width Ramp (LeetCode 962)

Largest $j - i$ with $i < j$ and $\text{nums}[i] \leq \text{nums}[j]$. $[6, 0, 8, 2, 1, 5] \rightarrow 4$.

Idea. Only *strictly decreasing prefix minimums* can be useful left ends (a later, larger value to the right of a smaller one is never a better left end) $\rightarrow$ build that stack first; then scan j from the right and **pop** every candidate i that $\text{nums}[j]$ satisfies.

```
def max_width_ramp(nums):
    """962: max j - i with i < j and nums[i] <= nums[j]."""
    stack = []
    # candidate left ends: strictly
    # decreasing values
    for i, x in enumerate(nums):
        if not stack or x < nums[stack[-1]]:
            stack.append(i)
    best = 0
    for j in range(len(nums) - 1, -1, -1):
        # scan right ends from the right
        while stack and nums[stack[-1]] <= nums[j]:
            best = max(best, j - stack.pop())
    return best
```

132 Pattern (LeetCode 456)

Is there $i < j < k$ with $\text{nums}[i] < \text{nums}[k] < \text{nums}[j]$? $[3, 1, 4, 2] \rightarrow \text{true} (1, 4, 2)$.

Idea. Scan from the **right**, keeping a decreasing stack of candidates for the "3" ($\text{nums}[j]$). Values popped by a larger x become the best "2" (third) — the largest value that is smaller than something to its left. Any later (leftward) $x < \text{third}$ is a valid "1".

```
def find_132_pattern(nums):
    """456: exists i<j<k with nums[i] < nums[k] < nums[j]? Scan from the right."""
    stack = [] # decreasing stack of candidates for
    '3'
    third = float('-inf') # the best '2' seen so far (popped
values)
    for x in reversed(nums):
        if x < third:
            return True # x plays '1'
        while stack and stack[-1] < x:
            third = stack.pop() # popped value is smaller than x ->
a valid '2' below a '3'
        stack.append(x)
    return False
```

5.12 Monotonic stack for lexicographic minimisation ("remove to make it smallest")

Recognition. "Delete k items (or choose a subsequence) to make the result as small/large as possible in dictionary order." — **a bigger character before a smaller one is wasteful, so pop it.**

Remove K Digits (LeetCode 402)

num = "1432219", k = 3 → "1219". "10200", k = 1 → "200". "10", k = 2 → "0".

Derive it. Walk left to right. If the new digit is **smaller** than the top of the stack, the top digit would make the number bigger than necessary → **pop it** (this uses one removal). If we still have removals left at the end, remove from the **end** (the stack is non-decreasing there, so the largest digits are last). Strip leading zeros; empty → "0".

digit	pops (bigger digit before a smaller one)	stack	removals left
1	-	1	3
4	-	14	3
3	pop 4	13	2
2	pop 3	12	1
2	-	122	1
1	pop 2	121	0
9	-	1219	0

Result = **1219**

```
def remove_k_digits(num, k):
    stack = []
    for d in num:
        while k and stack and stack[-1] > d:
            stack.pop()
            # a bigger digit BEFORE a smaller
            # one is wasteful
        k -= 1
        stack.append(d)
    if k:
        stack = stack[: -k]
        # still need to remove: drop from
        # the (largest) end
    res = ''.join(stack).rstrip('0')
    return res or '0'
```

Remove Duplicate Letters / Smallest Subsequence of Distinct Characters (LeetCode 316 / 1081)

"cbacdcbc" → "acdb"; "bcabc" → "abc". Each letter exactly once, lexicographically smallest.

Idea. Same greedy, with two extra rules: skip a letter already in the result, and only pop the top if it **appears again later** (`last[top] > i`) — otherwise popping would lose it forever.

```
def remove_duplicate_letters(s):
    last = {c: i for i, c in enumerate(s)}
    stack, seen = [], set()
    for i, c in enumerate(s):
        if c in seen:
            continue
        while stack and stack[-1] > c and last[stack[-1]] > i:
            # top is bigger AND appears again
            # later
            seen.discard(stack.pop())
        stack.append(c)
        seen.add(c)
    return ''.join(stack)
```

Find the Most Competitive Subsequence (LeetCode 1673)

Smallest subsequence of length `k`: `[3,5,2,6]`, `k=2` → `[2,6]`.

Idea. Pop while the top is bigger **and enough elements remain** to still fill `k` slots.

```

def most_competitive(nums, k):
    """1673: lexicographically smallest subsequence of length k."""
    stack = []
    n = len(nums)
    for i, x in enumerate(nums):
        while stack and stack[-1] > x and len(stack) - 1 + (n - i) >= k:  # enough elements left to
            still fill k
            stack.pop()
        if len(stack) < k:
            stack.append(x)
    return stack

```

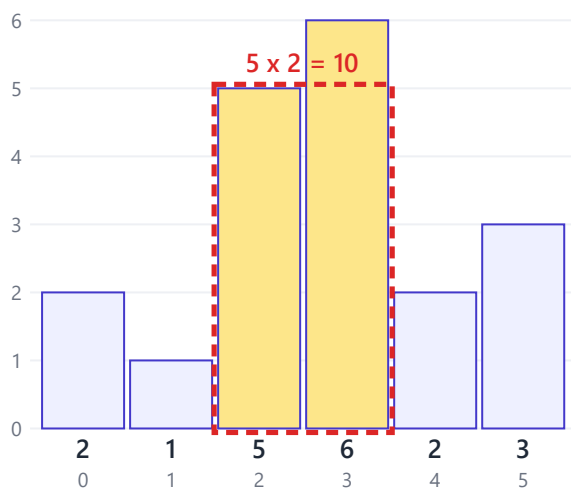
Part 6 · Monotonic Stack for Areas and Contributions

Recognition. "How far can this element extend before something smaller (or larger) stops it?" — i.e. you need **both** the previous-smaller and the next-smaller boundary of every element. One pass of a monotonic stack gives both: **the arriving element is the right boundary of everything it pops, and the element exposed underneath is the left boundary.**

6.1 Largest Rectangle in Histogram (LeetCode 84)

heights = [2, 1, 5, 6, 2, 3] → **10** (the bars of height 5 and 6 form a 5×2 rectangle).

Largest Rectangle in Histogram: heights [2, 1, 5, 6, 2, 3]



Best rectangle: height 5 over the two bars of heights 5 and 6 (indices 2..3) → area 10.

The key difficulty. If bar 5 is going to be the height of the rectangle, how far can it extend?

left until a bar smaller than 5
right until a bar smaller than 5

smaller smaller
↓ ↓
[2, 1, 5, 6, 2, 3]
← 5 →

So the problem becomes: **for each bar find the boundaries where a smaller bar stops it** — exactly the "waiting for a smaller element" problem.

Derive it. When the 2 at index 4 arrives it is smaller than both 6 and 5:

- it tells us 6 can no longer extend past here → pop 6;
- it tells us 5 can no longer extend past here either → pop 5.

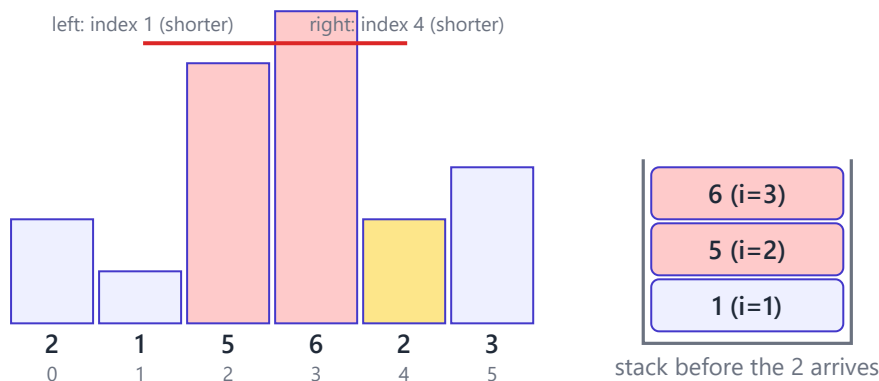
(A note on the session's wording: the reason a bar is popped is that the arriving bar is shorter than it — $2 < 6$, then $2 < 5$ — not that the arriving bar "violates $5 < 6$ ".)

After popping 6, what does the stack tell us about the left boundary of 5? The bar underneath 5 on the stack is 1, which is smaller than 5. So 5 can extend from just after index 1 through index 3:

height = 5, width = right - left - 1 = 4 - 1 - 1 = 2, area = 10

When a bar is popped, the element now exposed underneath tells us its LEFT boundary; the current (smaller) bar tells us its RIGHT boundary. That is the entire trick.

When the 2 at index 4 arrives, the taller bars on the stack are FINISHED



pop 6: left = 5 (new top, i=2) -> width = 4 - 2 - 1 = 1 -> area 6

pop 5: left = 1 (new top, i=1) -> width = 4 - 1 - 1 = 2 -> area 10

Popped bar = the height; the bar exposed underneath = left boundary; the arriving bar = right boundary.

The invariant. Before the 2 arrives the stack is [1, 5, 6] → **increasing** heights, bottom → top. Why? Each bar on the stack is saying "I can still extend to the right because everything I've seen so far is $\geq$ me." When a smaller bar arrives that promise breaks for every taller bar on top, and they are finished.

Maintain an increasing stack of bars that can still extend to the right.

current < stack_top → the top bar is finished: compute its area

Store indices, not heights — we need positions for width = right - left - 1.

The sentinel trick. After the last bar, bars still on the stack extend to the end. Appending a height-0 bar forces every remaining bar to be popped and measured, so no cleanup loop is needed.

Full trace

i: height	pops (each pop finalises one bar)	stack after push	best area
0: 2	-	[2(i=0)]	0
1: 1	pop bar 2 (i=0): left=-1, right=1 → width 1 → area 2	[1(i=1)]	2
2: 5	-	[1(i=1), 5(i=2)]	2
3: 6	-	[1(i=1), 5(i=2), 6(i=3)]	2
4: 2	pop bar 6 (i=3): left=2, right=4 → width 1 → area 6 pop bar 5 (i=2): left=1, right=4 → width 2 → area 10	[1(i=1), 2(i=4)]	10
5: 3	-	[1(i=1), 2(i=4), 3(i=5)]	10

i: height	pops (each pop finalises one bar)	stack after push	best area
6: 0 (sentinel)	pop bar 3 (i=5): left=4, right=6 → width 1 → area 3 pop bar 2 (i=4): left=1, right=6 → width 4 → area 8 pop bar 1 (i=1): left=-1, right=6 → width 6 → area 6	[0(i=6)]	10

Largest rectangle = **10**

```
def largest_rectangle_area(heights):
    stack = []                                # indices, heights INcreasing bottom
    -> top
    best = 0
    heights = heights + [0]                  # sentinel: a height-0 bar flushes
    the stack
    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] > h:    # h is shorter: the top bar cannot
            extend further right
            height = heights[stack.pop()]
            left = stack[-1] if stack else -1        # first shorter bar on the left
            (exposed by the pop)
            width = i - left - 1                    # i is the first shorter bar on the
            right
            best = max(best, height * width)
            stack.append(i)
    return best
```

Complexity. Every bar is pushed once and popped once → **O(n)** time, **O(n)** space.

Mental model. A monotonic increasing stack of bars that can still grow rightwards; popping a bar means its maximum width has just been determined.

6.2 Maximal Rectangle (LeetCode 85)

A binary matrix of '0' / '1'; find the largest rectangle containing only 1s.

Idea (reduce to the histogram). Process the matrix **row by row**; for each column keep the **height of consecutive 1s ending at this row** (reset to 0 at a '0'). Every row is then a histogram → apply §6.1. Total **O(rows × cols)**.

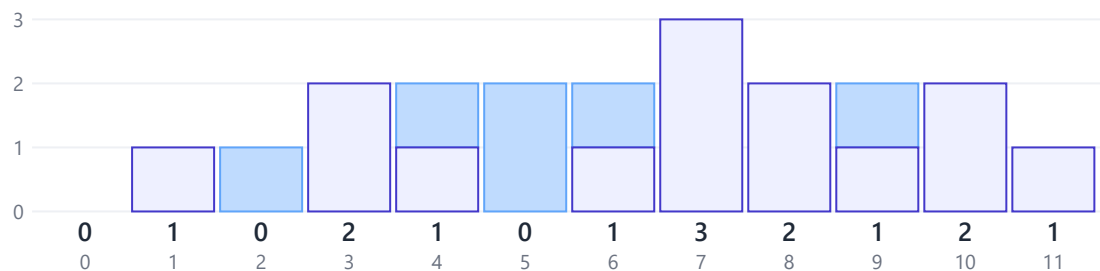
```
row 0: 1 0 1 0 0   heights [1,0,1,0,0]
row 1: 1 0 1 1 1   heights [2,0,2,1,1]
row 2: 1 1 1 1 1   heights [3,1,3,2,2]   ← best rectangle here has area 6
row 3: 1 0 0 1 0   heights [4,0,0,3,0]
```

```
def maximal_rectangle(matrix):
    if not matrix:
        return 0
    cols = len(matrix[0])
    heights = [0] * cols
    best = 0
    for row in matrix:
        for c in range(cols):
            heights[c] = heights[c] + 1 if row[c] == '1' else 0 # histogram of consecutive 1s
            # ending at this row
        best = max(best, largest_rectangle_area(heights[:]))
    return best
```

6.3 Trapping Rain Water (LeetCode 42)

`[0,1,0,2,1,0,1,3,2,1,2,1] → 6.`

Trapping Rain Water: level at each bar = $\min(\text{tallest to the left, tallest to the right})$



Blue = trapped water. Total = 6 units.

Two ways to think about it.

- By column:** water above a bar = $\min(\text{tallest to its left, tallest to its right}) - \text{height}$. (DP with prefix/suffix maxima, or **two pointers** in $O(1)$ space.)
- By pool (stack):** water sits in a *pool* whose floor is a bar, closed by a taller wall on each side. Keep a **decreasing** stack of walls; when a taller bar `x` arrives, **pop the floor** — the new top is the left wall — and add the pool $\text{width} \times \text{depth}$, where

```
width = i - left_wall_index - 1
depth = min(x, left_wall_height) - floor_height
```

i: height	pools closed	stack (decreasing walls)	water so far
0: 0	-	[0(i=0)]	0
1: 1	floor 0: no left wall	[1(i=1)]	0
2: 0	-	[1(i=1), 0(i=2)]	0
3: 2	floor 0: width 1 × depth 1 = 1; floor 1: no left wall	[2(i=3)]	1

i: height	pools closed	stack (decreasing walls)	water so far
4: 1	-	[2(i=3), 1(i=4)]	1
5: 0	-	[2(i=3), 1(i=4), 0(i=5)]	1
6: 1	floor 0: width 1 × depth 1 = 1	[2(i=3), 1(i=4), 1(i=6)]	2
7: 3	floor 1: width 2 × depth 0 = 0; floor 1: width 3 × depth 1 = 3; floor 2: no left wall	[3(i=7)]	5
8: 2	-	[3(i=7), 2(i=8)]	5
9: 1	-	[3(i=7), 2(i=8), 1(i=9)]	5
10: 2	floor 1: width 1 × depth 1 = 1	[3(i=7), 2(i=8), 2(i=10)]	6
11: 1	-	[3(i=7), 2(i=8), 2(i=10), 1(i=11)]	6

Total water = 6

```
def trap_rain_water(height):
    stack = []                                # indices of a decreasing sequence
    of walls
    water = 0
    for i, h in enumerate(height):
        while stack and h > height[stack[-1]]:
            bottom = height[stack.pop()]       # the "floor" of a pool
            if not stack:
                break                           # no left wall: water would run out
            width = i - stack[-1] - 1
            bounded = min(h, height[stack[-1]]) - bottom  # water level minus the floor
            water += width * bounded
        stack.append(i)
    return water
```

The O(1)-space two-pointer version for comparison:

```
def trap_rain_water_two_pointers(height):
    l, r = 0, len(height) - 1
    left_max = right_max = water = 0
    while l < r:
        if height[l] < height[r]:
            left_max = max(left_max, height[l])
            water += left_max - height[l]
            l += 1
        else:
            right_max = max(right_max, height[r])
            water += right_max - height[r]
            r -= 1
    return water
```

Complexity. Stack: $O(n)$ time / $O(n)$ space. Two pointers: $O(n)$ / $O(1)$. *The stack method is worth knowing because it is the same "pop = a finished region" idea as the histogram.*

6.4 Sum of Subarray Minimums (LeetCode 907)

`[3, 1, 2, 4] → 17` (sum of the minimum of every contiguous subarray).

The contribution technique. Instead of enumerating $O(n^2)$ subarrays, ask: **for each element, in how many subarrays is it the minimum?** If `arr[i]` is the minimum of a subarray, that subarray's left end lies in `(prev_smaller, i]` and its right end in `[i, next_smaller)`:

```
number of subarrays where arr[i] is the min = left[i] × right[i]
contribution = arr[i] × left[i] × right[i]
```

- `left[i] = i - previous_smaller_index`
- `right[i] = next_smaller_index - i`

Both come from monotonic stacks.

Ties. With equal values, one subarray could be counted for several elements. Break ties consistently: use **strictly** smaller on one side and **smaller-or-equal** on the other, so each subarray is attributed to exactly **one** (the leftmost) minimum.

i	value	choices for left end	choices for right end	contribution = value × left × right
0	3	1	1	3
1	1	2	3	6
2	2	1	2	4
3	4	1	1	4

Sum of subarray minimums = **17**

```

def sum_subarray_mins(arr):
    """907: sum of min(subarray) over all subarrays (mod 1e9+7). Contribution technique."""
    MOD = 10**9 + 7
    n = len(arr)
    left = [0] * n           # number of choices for the left end
    right = [0] * n          # number of choices for the right
    end
    stack = []
    for i in range(n):
        while stack and arr[stack[-1]] > arr[i]:           # strictly greater: bars to the left
            that are larger
            stack.pop()
            left[i] = i - (stack[-1] if stack else -1)
            stack.append(i)
        stack = []
        for i in range(n - 1, -1, -1):
            while stack and arr[stack[-1]] >= arr[i]:       # >= on ONE side breaks ties so
                nothing is counted twice
                stack.pop()
                right[i] = (stack[-1] if stack else n) - i
                stack.append(i)
    return sum(arr[i] * left[i] * right[i] for i in range(n)) % MOD

```

Complexity. $O(n)$.

Sum of Subarray Ranges (LeetCode 2104)

Sum of $(\max - \min)$ over all subarrays. Since $\sum (\max - \min) = \sum \max - \sum \min$, run the same contribution technique twice — once for minimums and once for maximums.

```

def sum_subarray_ranges(nums):
    """2104: sum of (max - min) over all subarrays = (sum of maxes) - (sum of mins)."""
    def total(cmp_left, cmp_right, arr):
        n = len(arr)
        left, right, stack = [0] * n, [0] * n, []
        for i in range(n):
            while stack and cmp_left(arr[stack[-1]], arr[i]):
                stack.pop()
            left[i] = i - (stack[-1] if stack else -1)
            stack.append(i)
        stack = []
        for i in range(n - 1, -1, -1):
            while stack and cmp_right(arr[stack[-1]], arr[i]):
                stack.pop()
            right[i] = (stack[-1] if stack else n) - i
            stack.append(i)
        return sum(arr[i] * left[i] * right[i] for i in range(n))
    sum_min = total(lambda a, b: a > b, lambda a, b: a >= b, nums)
    sum_max = total(lambda a, b: a < b, lambda a, b: a <= b, nums)
    return sum_max - sum_min

```

Part 7 · Stack of Groups — Car Fleet

7.1 Car Fleet (LeetCode 853)

Cars travel toward the same destination. `position[i]` = start, `speed[i]` = speed, `target` = destination. A faster car behind a slower one catches it and they become **one fleet** travelling at the slower car's speed (cars cannot pass). How many fleets arrive? `target = 12`, `position = [10, 8, 0, 5, 3]`, `speed = [2, 4, 1, 1, 3]` → **3**.

Step 1 — time to reach the target for each car: $(\text{target} - \text{position}) / \text{speed}$.

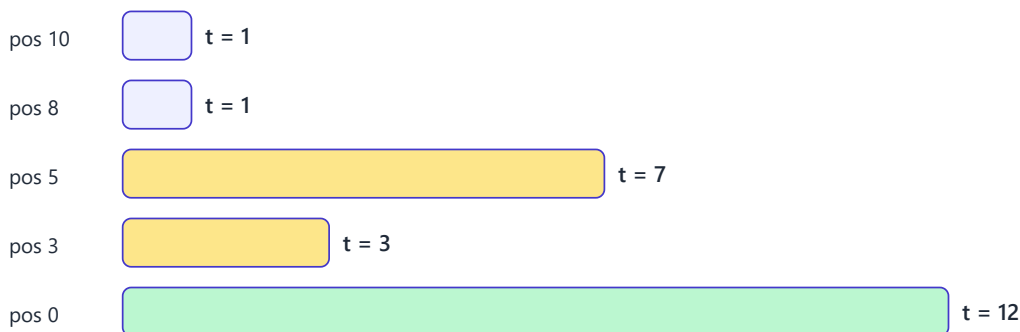
```
car at 10, speed 2 → (12-10)/2 = 1
car at 8, speed 4 → (12- 8)/4 = 1
car at 0, speed 1 → (12- 0)/1 = 12
car at 5, speed 1 → (12- 5)/1 = 7
car at 3, speed 3 → (12- 3)/3 = 3
```

Step 2 — sort by position, closest to the target first. Cars can't pass, so only the car directly ahead (or the fleet it belongs to) matters:

```
front ↓
position: 10    8    5    3    0
time:      1    1    7    3   12
```

Step 3 — the key observation. Looking from the front toward the back: *a car behind catches the fleet ahead if its arrival time is smaller or equal. If it takes longer, it can never catch up and forms a new fleet**.*

Car Fleet: cars from front to back; bar = time to reach the target



Stack of fleet times: [1, 7, 12] → **3 fleets**

pos 8 ($t=1 \leq 1$) merges with pos 10; pos 3 ($t=3 \leq 7$) merges with pos 5; pos 0 ($t=12 > 7$) starts a new fleet.

The stack holds **the arrival time of each fleet** (not individual cars):

```
if current time > stack top → cannot catch the fleet ahead → NEW fleet (push)
if current time <= stack top → catches it → same fleet (do nothing)
```

Notice we do **not** change the fleet's time when a car merges: once the car catches the fleet, they travel together and arrive with the **slower** fleet's time — the time already on the stack.

car (front → back)	time to target	decision	stack of fleet times
pos 10, speed 2	1	$t > \text{top of stack}$: cannot catch the fleet ahead → NEW fleet	[1]
pos 8, speed 4	1	$t \leq \text{top}$: catches up → merges into the fleet ahead	[1]
pos 5, speed 1	7	$t > \text{top of stack}$: cannot catch the fleet ahead → NEW fleet	[1, 7]
pos 3, speed 3	3	$t \leq \text{top}$: catches up → merges into the fleet ahead	[1, 7]
pos 0, speed 1	12	$t > \text{top of stack}$: cannot catch the fleet ahead → NEW fleet	[1, 7, 12]

Fleets = 3

Second example (a drill from the session). `target = 10`, `position = [6, 8, 3, 1]`, `speed = [3, 2, 1, 2]`.
Sorted by position: `(8, ·)`, `(6, ·)`, `(3, ·)`, `(1, ·)`.

Correction. The session gave the car at position 8 the time `0.66` and marked it "correct". It is $(10 - 8) / 2 = 1$. The other three times (`1.33`, `7`, `4.5`) were right, and the final answer (3 fleets) is unchanged: `1` → `1.33` > `1` → new fleet → `7` > `1.33` → new fleet → `4.5` ≤ `7` → merges.

car (front → back)	time to target	decision	stack of fleet times
pos 8, speed 2	1	$t > \text{top of stack}$: cannot catch the fleet ahead → NEW fleet	[1]
pos 6, speed 3	1.33333	$t > \text{top of stack}$: cannot catch the fleet ahead → NEW fleet	[1, 1.33333]
pos 3, speed 1	7	$t > \text{top of stack}$: cannot catch the fleet ahead → NEW fleet	[1, 1.33333, 7]
pos 1, speed 2	4.5	$t \leq \text{top}$: catches up → merges into the fleet ahead	[1, 1.33333, 7]

Fleets = 3

```
def car_fleet(target, position, speed):
    cars = sorted(zip(position, speed), reverse=True)           # closest to the target first
    stack = []                                                  # arrival times of the fleets
    for pos, spd in cars:
        t = (target - pos) / spd
        if not stack or t > stack[-1]:
            stack.append(t)                                     # cannot catch the fleet ahead: a
new fleet
        # else: t <= stack[-1] -> it catches up and merges into the fleet ahead (its time is
unchanged)
    return len(stack)
```

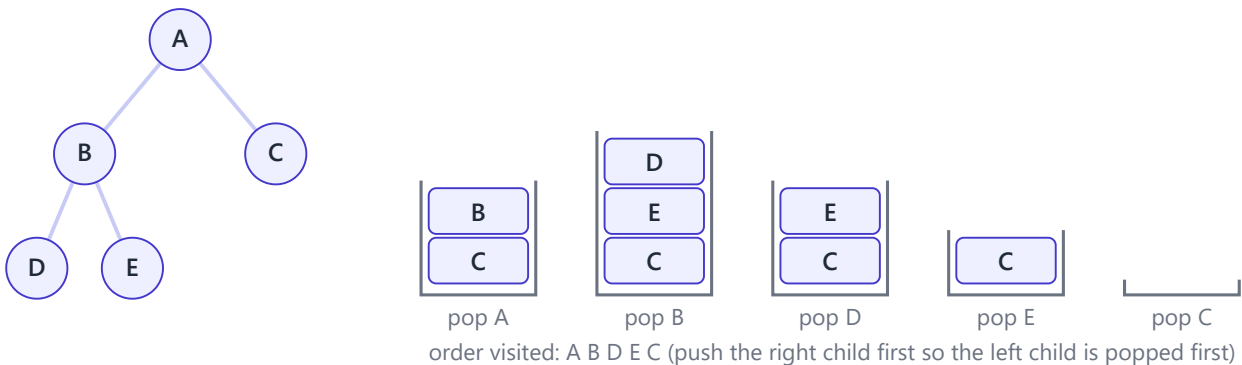
Complexity. Sorting $O(n \log n)$ + a single pass. Because we only ever compare with the top, the stack could be replaced by one variable (`prev_time`); the stack version makes the "fleets ahead" idea explicit.

Why it is a stack problem. *Unresolved groups* (fleets that might still be caught) wait on the stack; the newest one (nearest behind) is compared first; the rule is a monotonic one — the times on the stack strictly increase from the front fleet to the back fleet.

Part 8 · The Stack as the Call Stack — DFS and Traversals

Recursion **is** a stack: each call pushes a frame, each return pops it (see §1.3). Any recursive algorithm can be rewritten with an **explicit stack** — useful to avoid `RecursionError` on deep inputs and to gain control (pause, resume, custom order).

Iterative DFS: the explicit stack replaces the recursion



8.1 Iterative DFS on a graph

Idea. Push the start; repeat: pop a node, visit it if new, push its neighbours. (*Push neighbours in reverse if you want them visited in listed order.*)

```
def dfs_iterative(graph, start):
    visited, order, stack = set(), [], [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
        for nb in reversed(graph[node]):
            if nb not in visited:
                stack.append(nb)
    return order
```

reversed so neighbours are visited in listed order

8.2 Binary tree traversals without recursion

In-order (LeetCode 94) — left, node, right. Go as far left as possible pushing nodes; pop → visit → move to the right child.

step	stack	visited
go left, push 4	[4]	[]

step	stack	visited
go left, push 2	[4, 2]	[]
go left, push 1	[4, 2, 1]	[]
pop → visit 1	[4, 2]	[1]
pop → visit 2	[4]	[1, 2]
go left, push 3	[4, 3]	[1, 2]
pop → visit 3	[4]	[1, 2, 3]
pop → visit 4	[]	[1, 2, 3, 4]
go left, push 5	[5]	[1, 2, 3, 4]
pop → visit 5	[]	[1, 2, 3, 4, 5]

```
def inorder_iterative(root):
    result, stack, node = [], [], root
    while node or stack:
        while node:                                # go as far left as possible
            stack.append(node)
            node = node.left
        node = stack.pop()
        result.append(node.val)                     # visit
        node = node.right                           # then the right subtree
    return result

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

Pre-order (LeetCode 144) — node, left, right. Pop, visit, push **right first** then left (so left is popped first).

```
def preorder_iterative(root):
    if not root:
        return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)                     # visit
        if node.right:
            stack.append(node.right)                # right first ...
        if node.left:
            stack.append(node.left)                  # ... so left is popped (visited)
    first
    return result
```

Post-order (LeetCode 145) — left, right, node. Trick: produce *node, right, left* (a mirrored pre-order) and **reverse** it.

```
def postorder_iterative(root):
    if not root:
        return []
    result, stack = [], [root]
    while stack:
        node = stack.pop()
        result.append(node.val)           # root, right, left ...
        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)
    return result[::-1]                  # ... reversed = left, right, root
```

8.3 Number of Islands with an explicit stack (LeetCode 200)

Flood-fill each island with a stack of cells instead of recursion (no stack overflow on a huge grid).

```
def num_islands_iterative(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    seen = [[False] * cols for _ in range(rows)]
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and not seen[r][c]:
                count += 1
                stack = [(r, c)]
                seen[r][c] = True
                while stack:
                    # explicit stack instead of
                    # recursion (no stack overflow)
                    x, y = stack.pop()
                    for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                        nx, ny = x + dx, y + dy
                        if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == '1' and not
                        seen[nx][ny]:
                            seen[nx][ny] = True
                            stack.append((nx, ny))
    return count
```

Recursion	Explicit stack
implicit call stack; concise	explicit <code>stack</code> ; no recursion limit
stack overflow on very deep input	limited only by memory
order comes "for free"	you control push order (right before left, etc.)

Part 9 · Related: the Monotonic Deque

Recognition. A *sliding window* where you need the **maximum (or minimum) of every window**. A **monotonic stack*** only removes from one end; a window also has elements falling out of the front, so we need a deque** (double-ended queue).

9.1 Sliding Window Maximum (LeetCode 239)

```
nums = [1,3,-1,-3,5,3,6,7], k = 3 → [3,3,5,5,6,7].
```

Derive it. Keep the **indices** of window elements in a deque whose *values are decreasing* from front to back:

- **back:** a new element x makes every smaller element before it useless (they leave the window earlier *and* are smaller) → pop them from the **back**;
- **front:** if the front index has slid out of the window ($\leq i - k$) → pop it from the **front**;
- the **front is always the window maximum**.

```
from collections import deque

def max_sliding_window(nums, k):
    dq = deque()                                # indices; values DEcreasing front -
    > back
    result = []
    for i, x in enumerate(nums):
        while dq and nums[dq[-1]] <= x:
            dq.pop()                            # smaller elements behind x can
never be a maximum
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()                        # the front slid out of the window
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

Complexity. Each index enters and leaves the deque once → **$O(n)$** (better than the heap solution's $O(n \log n)$).

Part 10 · Templates, Decision Guide and Cheat Sheets

10.1 The templates to memorise

These are the templates worth memorising — not individual solutions. Names such as `can_resolve` or `condition` stand for problem-specific logic.

Template 1 — Matching / nesting (brackets)

```
pairs = {'(': ')', '[': ']', '{': '}', '(': ')', '[': ']', '{': '}'
stack = []
for ch in s:
    if ch in pairs:
        if not stack or stack[-1] != pairs[ch]:
            return False
        stack.pop()
    else:
        stack.append(ch)
return not stack
```

closer
opener
leftovers = unmatched openers

Template 2 — Cancelling (adjacent duplicates, backspace, collisions)

```
stack = []
for x in items:
    while stack and cancels(stack[-1], x):
        stack.pop()
    stack.append(x)
```

top and newcomer annihilate
(asteroids also need "does the newcomer survive?")
(skip the push if the newcomer was destroyed)

Template 3 — Evaluate with a stack of terms / operands

```
# RPN: operands are pushed, an operator pops two (RIGHT operand first)
b = stack.pop(); a = stack.pop(); stack.append(a OP b)

# Calculator II: the stack holds signed TERMS; '*' and '/' fold into the last term
if op == '+': stack.append(num)
elif op == '-': stack.append(-num)
elif op == '*': stack.append(stack.pop() * num)
elif op == '/': stack.append(int(stack.pop() / num))
answer = sum(stack)
```

Template 4 — Save / restore context on nesting ((, [, call)

```
stack = []
cur = initial_state
for token in tokens:
    if token == OPEN:
        stack.append(cur)          # suspend what we were doing
        cur = initial_state
    elif token == CLOSE:
        prev = stack.pop()        # resume
        cur = combine(prev, cur)
    else:
        cur = update(cur, token)
```

Template 5 — Monotonic stack, Form A (current resolves older elements)

```
answer = [-1] * n
stack = []                      # indices
for i in range(n):
    while stack and nums[i] > nums[stack[-1]]:  # Next Greater (use < for Next Smaller)
        j = stack.pop()
        answer[j] = nums[i]                  # or i - j for "how many days"
    stack.append(i)
# whatever remains in the stack has no answer → keeps -1
```

Template 6 — Monotonic stack, Form B (current asks for its answer)

```
answer = [-1] * n
stack = []
for i in range(n):
    while stack and nums[stack[-1]] >= nums[i]:  # Previous Smaller (use <= for Previous Greater)
        stack.pop()                             # useless candidates
    if stack:
        answer[i] = nums[stack[-1]]
    stack.append(i)
```

Template 7 — Largest rectangle / boundaries with a sentinel

```
heights.append(0)                # sentinel flushes the stack
stack = []
for i, h in enumerate(heights):
    while stack and heights[stack[-1]] > h: # h is shorter: the top bar is finished
        height = heights[stack.pop()]
        left = stack[-1] if stack else -1   # exposed element = left boundary
        width = i - left - 1               # current index = right boundary
        best = max(best, height * width)
    stack.append(i)
```

Template 8 — Contribution technique (left/right counts)

```
# left[i] = i - previous_smaller_index      (pop while stack top > arr[i])
# right[i] = next_smaller_index - i        (pop while stack top >= arr[i])  <- ties on ONE
side only
total = sum(arr[i] * left[i] * right[i] for i in range(n))
```

Template 9 — Design: store extra info with every element

```
stack.append((x, min(x, stack[-1][1]) if stack else x)) # (value, min so far)
get_min = stack[-1][1]
```

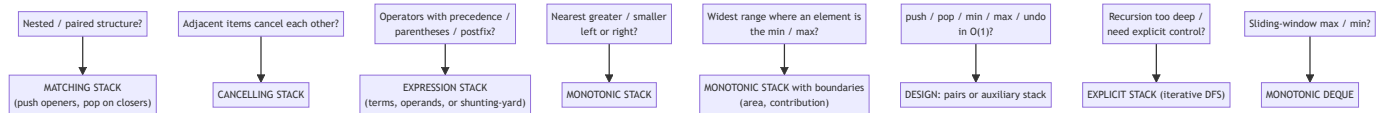
Template 10 — Iterative DFS / tree traversal

```
stack = [root]
while stack:
    node = stack.pop()
    visit(node)
    for child in reversed(children(node)): # reverse so the first child is popped first
        stack.append(child)
```

Template 11 — Monotonic deque (sliding window extreme)

```
dq = deque()                # indices, values decreasing front → back
for i, x in enumerate(nums):
    while dq and nums[dq[-1]] <= x: dq.pop() # smaller elements behind x are useless
    dq.append(i)
    if dq[0] <= i - k: dq.popleft()        # front slid out of the window
    if i >= k - 1: result.append(nums[dq[0]])
```

10.2 Final decision guide



If you see...	Use
brackets, tags, nested calls, "valid ..."	matching stack
"remove adjacent duplicates", <code>#</code> = backspace, collisions	cancelling stack
<code>+</code> <code>-</code> <code>*</code> <code>/</code> , parentheses, RPN, encoded strings, paths	expression / context stack
"next greater", "days until warmer", "span", "next smaller"	monotonic stack (Form A)
"nearest smaller to the left", "previous greater"	monotonic stack (Form B)
rectangles in histograms, trapped water, sum of subarray minimums	monotonic stack with left & right boundaries
"make the number/string smallest by deleting k"	monotonic stack (greedy pop)
<code>min()</code> / <code>max()</code> in $O(1)$ on a stack, queue from stacks	design with pairs / two stacks
groups merge when one catches up	stack of groups (Car Fleet)
DFS, in/pre/post-order without recursion	explicit stack
sliding-window maximum	monotonic deque

The biggest thing to retain is not the syntax. Ask: "What is waiting, and what event resolves it?" Then: *Do newer waiting elements get resolved first?* → *stack*. What invariant does the stack maintain? What does POP mean?

10.3 Which comparison? — monotonic cheat sheet

Want	Direction	Pop while	Stack (bottom → top)	Form
Next greater	→	<code>nums[i] > nums[top]</code>	decreasing	A (answer at pop)
Next smaller	→	<code>nums[i] < nums[top]</code>	increasing	A
Previous greater	→ scan, look back	<code>nums[top] <= nums[i]</code>	decreasing	B (answer = top)
Previous smaller	→ scan, look back	<code>nums[top] >= nums[i]</code>	increasing	B
Largest rectangle	→	<code>heights[top] > h</code>	increasing	A + width from boundaries
Trapping rain water	→	<code>height[top] < h</code>	decreasing	A (pool closes at pop)
Remove K digits / smallest subsequence	→	<code>top > digit</code> (while removals left)	non-decreasing	greedy

10.4 Complexity cheat sheet

Problem	Time	Space
Valid Parentheses / Min Remove / Longest Valid	$O(n)$	$O(n)$
Min Add to Make Valid	$O(n)$	$O(1)$
RPN / Calculator I & II / Decode String / Simplify Path	$O(n)$	$O(n)$
Min Stack / Max Stack (all ops)	$O(1)$	$O(n)$
Queue from two stacks	amortized $O(1)$ per op	$O(n)$
Next Greater / Daily Temperatures / Stock Span	$O(n)$	$O(n)$
Circular Next Greater	$O(n)$	$O(n)$
Remove K Digits / Remove Duplicate Letters	$O(n)$	$O(n)$
Largest Rectangle in Histogram	$O(n)$	$O(n)$
Maximal Rectangle	$O(\text{rows} \times \text{cols})$	$O(\text{cols})$
Trapping Rain Water (stack / two pointers)	$O(n)$ / $O(n)$	$O(n)$ / $O(1)$
Sum of Subarray Minimums / Ranges	$O(n)$	$O(n)$
Car Fleet	$O(n \log n)$	$O(n)$
Asteroid Collision	$O(n)$	$O(n)$
Iterative DFS / tree traversal	$O(V + E)$ / $O(n)$	$O(V)$ / $O(h)$
Sliding Window Maximum (deque)	$O(n)$	$O(k)$

10.5 Pitfalls checklist

Pitfall	Fix
<code>pop()</code> / <code>[-1]</code> on an empty stack	guard with <code>while stack</code> and <code>...</code> / <code>if stack</code>
Comparing indices instead of values	<code>nums[stack[-1]]</code> , not <code>stack[-1]</code>
Forgetting the leftovers	initialise answers to <code>-1</code> / <code>0</code> , or flush with a sentinel
Wrong pop direction ("pop when it does not help")	pop when the current element does resolve the top
<code><</code> vs <code><=</code> on duplicates	decide strict vs non-strict; for counting problems use one strict + one non-strict side
RPN operand order	first pop = right operand
Calculator: applying the operator you just read	apply the previous operator when a number finishes
Division truncation	<code>int(a / b)</code> (toward zero), not <code>//</code> (floors negatives)
<code>pop(0)</code> / <code>insert(0, x)</code> on a list	use the end of the list, or <code>collections.deque</code>
Deep recursion	<code>RecursionError</code> → explicit stack
Post-order iterative	build <i>node</i> , <i>right</i> , <i>left</i> and reverse

Pitfall	Fix
Storing values when positions are needed (spans, widths)	store indices
Not resetting/merging state on <code>]</code> / <code>)</code> (nested parsers)	push the <i>whole context</i> on <code>[</code> , restore on <code>]</code>

10.6 Which patterns combine?

Problem	Patterns
Maximal Rectangle	rows → histogram → monotonic stack
Sum of Subarray Minimums / Ranges	monotonic stack × 2 (both boundaries) + contribution counting
Trapping Rain Water	monotonic stack (or two pointers)
Remove Duplicate Letters	monotonic stack + <code>last</code> -occurrence table + <code>seen</code> set
Stock Span	Previous Greater with counts
Sliding Window Maximum	monotonic <i>deque</i> (stack + window eviction)
Basic Calculator	context stack (save on <code>(</code>)
Asteroid Collision	cancelling stack + direction test

Part 11 · Practice, Recognition Drills and Interview Q&A

11.1 Practice list (LeetCode)

Solved in this guide

Pattern	Problems
Fundamentals / design	155 Min Stack · 232 Implement Queue using Stacks · 225 Implement Stack using Queues · 1472 Design Browser History · 1381 Design a Stack With Increment Operation · 946 Validate Stack Sequences · 682 Baseball Game
Matching & cancelling	20 Valid Parentheses · 921 Minimum Add to Make Valid · 1249 Minimum Remove to Make Valid · 32 Longest Valid Parentheses · 856 Score of Parentheses · 1190 Reverse Substrings Between Each Pair of Parentheses · 678 Valid Parenthesis String · 1047 Remove All Adjacent Duplicates · 844 Backspace String Compare · 735 Asteroid Collision
Expressions & parsing	150 Evaluate Reverse Polish Notation · 227 Basic Calculator II · 224 Basic Calculator · 394 Decode String · 71 Simplify Path · 636 Exclusive Time of Functions
Monotonic stack	496 Next Greater Element I · 503 Next Greater Element II · 739 Daily Temperatures · 901 Online Stock Span · 1019 Next Greater Node in Linked List · 1762 Buildings With an Ocean View · 962 Maximum Width Ramp · 456 132 Pattern · 402 Remove K Digits · 316 / 1081 Remove Duplicate Letters · 1673 Most Competitive Subsequence
Areas & contributions	84 Largest Rectangle in Histogram · 85 Maximal Rectangle · 42 Trapping Rain Water · 907 Sum of Subarray Minimums · 2104 Sum of Subarray Ranges
Stack of groups	853 Car Fleet
DFS / traversals	94 / 144 / 145 Binary Tree Inorder / Preorder / Postorder Traversal · 200 Number of Islands
Monotonic deque	239 Sliding Window Maximum

Further practice (suggestions, not solved here)

1544 Make The String Great · 1614 Maximum Nesting Depth of the Parentheses · 1021 Remove Outermost Parentheses · 1130 Minimum Cost Tree From Leaf Values · 2289 Steps to Make Array Non-decreasing · 1856 Maximum Subarray Min-Product · 769/768 Max Chunks To Make Sorted · 895 Maximum Frequency Stack · 716 Max Stack · 341 Flatten Nested List Iterator · 385 Mini Parser · 726 Number of Atoms · 1944 Number of Visible People in a Queue · 84 variants: 1793 / 2334.

11.2 Recognition drills

Cover the right-hand column and name the pattern first.

Scenario	Pattern
Are the brackets balanced and correctly nested?	matching stack
Repeatedly delete adjacent equal pairs	cancelling stack
Evaluate <code>["4", "13", "5", "/", "+"]</code>	RPN stack of operands

Scenario	Pattern
Evaluate $2*(3+4)-5/2$	expression stack / shunting-yard
Expand <code>3[a2[c]]</code>	context-save stack
Every day, how long until a warmer one?	monotonic stack (Form A, distance)
Nearest smaller value to the left of each element	monotonic stack (Form B)
Biggest rectangle under a skyline of bars	monotonic stack with left/right boundaries
Sum of the minimum over every subarray	monotonic stack + contribution counting
Delete <code>k</code> digits to make the smallest number	greedy monotonic stack
A stack whose minimum is available in $O(1)$	store <code>(value, min so far)</code>
Build a FIFO queue when you only have LIFO stacks	two stacks (inbox/outbox)
Cars that catch up merge into fleets	stack of fleet arrival times
Preorder traversal but recursion is forbidden	explicit stack
Maximum of every window of size <code>k</code>	monotonic deque

11.3 Interview questions and short answers

Q. What is a stack and where is it used? A LIFO structure: insert and remove at the same end. Used for undo, nested parsing, recursion (the call stack), DFS, expression evaluation, and "nearest previous/next" problems (monotonic stack).

Q. Why does a stack fit "next greater element"? Elements wait for a greater element; when a new element arrives it must resolve the **most recently waiting** ones first (they're on top) — LIFO. The stack keeps the unresolved elements in decreasing order, so the loop can stop at the first element it cannot resolve.

Q. Why is the monotonic stack $O(n)$ although it has a `while` inside a `for`? Every index is pushed once and popped at most once: total pops $\leq n$, so the total work is $O(n)$ (amortized $O(1)$ per element).

Q. Why store indices instead of values? To write the answer at the right position, and to compute distances/widths. Compare `nums[stack[-1]]`.

Q. How do you find the previous smaller element? Scan left $\rightarrow$ right; pop while `top >= current`; the remaining top (if any) is the answer; push current. The stack is increasing.

Q. Explain Largest Rectangle in Histogram. Maintain an increasing stack of bar indices. When a shorter bar arrives, pop taller bars; for each popped bar the new top is its left boundary and the current index is its right boundary: `width = i - left - 1`. A sentinel `0` flushes the stack. $O(n)$.

Q. How do you design a Min Stack with $O(1)$ `get_min`? Store `(value, minimum so far)` with each entry (or a parallel stack of minimums). Popping restores the previous minimum automatically.

Q. How can you implement a queue with two stacks? What is the complexity? Push to an inbox; pop from an outbox; when the outbox is empty, move everything from the inbox (reversing order). Each element moves at most twice $\rightarrow$ amortized $O(1)$.

Q. Why does RPN need no parentheses? Operators come after their operands, so the evaluation order is unambiguous; a stack of operands is enough.

Q. How do you convert infix to postfix? Shunting-yard: operands go to the output; an operator first pops higher-or-equal precedence operators (left-associative), then is pushed; `(` is pushed; `)` pops until `(`.

Q. Recursion vs an explicit stack? Recursion uses the call stack — concise but limited by recursion depth. An explicit stack has no such limit and lets you control the order; convert by pushing what the recursive call would have remembered.

Q. Stack vs deque, and when do you need a deque? A stack uses one end. A deque adds/removes at both ends — needed when elements also expire from the *front* (sliding-window maximum).

Q. `pop(0)` on a Python list — why is it a bug for a stack? It shifts every element: $O(n)$. Use the end of the list (or `collections.deque`).

11.4 One-page recap

- **Stack** = LIFO; push/pop/peek at the top are $O(1)$. In Python use a `list` (`append` / `pop` / `[-1]`), never the front.
- Core question: **"What is waiting, and what event resolves it? Does the newest waiting item resolve first?"**
- **Matching** — push openers, pop on a closer that matches; leftovers = invalid. **Cancelling** — pop when the top and the newcomer annihilate.
- **Expressions** — RPN: pop the *right* operand first; Calculator II: a stack of signed terms; nesting (`(` , `)`): save context, restore on the closer.
- **Design** — store (`value` , `extra`) per entry (Min Stack); queue = inbox + outbox → amortized $O(1)$.
- **Monotonic stack** — waiting elements kept in decreasing (next greater) or increasing (next smaller) order; pop while the current *resolves* the top; indices on the stack; leftovers = no answer. $O(n)$.
- **Form A** answers at *pop time*; **Form B** answers from the *top after popping*.
- **Areas** — arriving element = right boundary, exposed element = left boundary; sentinel `0` ; **contribution** = value $\times$ left $\times$ right with one strict side for ties.
- **Car Fleet** — sort by position from the front; new fleet iff `time > top` .
- **DFS** — the call stack made explicit; post-order = reversed (node, right, left).
- **Sliding-window extreme** — monotonic *deque*.